

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Facultad de Ingeniería — Ingeniería de Sistemas

CURSO:

WEB Y SISTEMAS MÓVILES

Momento 3 — Evaluación final

SISTEMA DE CONTROL DE ALQUILER DE MAQUINARIA DE CONSTRUCCIÓN

Documentación técnica de la versión integrada: patrón API Gateway, tres microservicios, cliente web y análisis teórico de tecnología móvil

INTEGRANTES:

MARTÍN ESCUDERO — ID: 919510

OVER BALLESTA — ID: 909215

JESÚS RIVERO — ID: 919420

DOCENTE:

ROYER MEJÍA

MONTERÍA, CÓRDOBA

6 de noviembre de 2026

Índice general

1 Generalidades del sistema **1**

1.1 Introducción . . . 2

1.2 Objetivo general y específicos . . . 3

1.3 Marco teórico . . . 4

1.4 Correcciones aplicadas de la retroalimentación del Momento 2 . . . 5

2 Definición del problema y requisitos **7**

2.1 Planteamiento del problema . . . 8

2.2 Alcance del sistema . . . 9

2.3 Requisitos funcionales . . . 10

2.4 Requisitos no funcionales . . . 12

3 Arquitectura y diseño del sistema **13**

3.1 Arquitectura de microservicios . . . 14

3.2 Diagrama de casos de uso . . . 16

3.3 Diagrama de clases del dominio . . . 18

3.4 Diagrama de componentes y estructura del código . . . 20

3.5 Roles del sistema: qué puede y qué no puede cada uno . . . 21

3.6 Los módulos del panel: cuáles son y qué hace cada uno . . . 22

4 Modelado y diseño de la base de datos **23**

4.1 Modelo entidad—relación y modelo relacional completo . . . 24

4.2 Normalización hasta la tercera forma normal . . . 27

4.3 Diccionario de datos . . . 29

4.4 Script de creación de la base de datos . . . 31

5 Especificación técnica de las piezas **35**

5.1 Cliente web (SPA en localhost:8443) . . . 36

5.2 API Gateway (Node/Express, 3000) . . . 37

5.3 API PHP (8000) . . . 38

5.4 API Node (3002) . . . 39

5.5 API Python (3003) . . . 40

5.6 Dueño de las tablas y librerías . . . 41

5.7 Contrato del API Gateway: ramas, dueños y métodos 42

6 Flujo del sistema paso a paso 43

6.1 Qué es la página y cómo se usa 44

6.2 El captcha: qué es y cómo funciona 46

6.3 El patrón API Gateway y su flujo 48

6.4 Flujo de crear alquiler y de devolución 49

6.5 Estados de la unidad y notificaciones 51

6.6 Evidencia viva: JSON del gateway explicado 53

6.7 Reglas exactas de cálculo del cobro (con ejemplo numérico) 55

7 Seguridad frente a ataques comunes 56

7.1 Arquitectura de seguridad en capas 57

7.2 Ataques comunes y contramedidas 58

7.3 Captcha y sesiones contra robots 59

7.4 Cierre de seguridad 60

8 Estrategia de pruebas 61

8.1 Herramientas y cobertura 62

8.2 Catalogo de casos 63

8.3 Escenarios límite, evidencia y rendimiento 64

8.4 Evidencia de la auditoría de dependencias 65

9 Despliegue y manual de ejecución 66

9.1 Requisitos de entorno 67

9.2 Variables de entorno 68

9.3 Guía de puesta en marcha 69

9.4 Salud de servicio y evidencia viva 70

9.5 Vistas de los módulos del panel en ejecución 72

9.6 Evidencia de la consola de arranque 75

10 Análisis teórico de tecnología móvil 77

10.1 Enfoques posibles 78

10.2 Por qué se eligió React Native con Expo 79

10.3 Prototipo entregado 80

10.4 El frontend de la web en un celular 81

10.5 Nota de integración consciente 85

11 Conclusiones **86**

11.1 Conclusiones 87

12 Referencias bibliográficas **88**

13 Anexos **90**

Índice de tablas

1 Observaciones del Momento 2 y su corrección en la versión final. 5
2 Requisitos funcionales completos del código entregado. 10
3 Requisitos no funcionales completos. 12
4 Puertos y responsables de cada pieza. 15
5 Qué puede hacer cada figura del sistema. 21
6 Módulos del panel, función, dueño de datos y quién lo usa. 22
7 Aplicación de las formas normales y respuesta a la retroalimentación. 27
8 Mecanismos compensatorios por no usar claves foráneas entre dominios. 27
9 Diccionario de datos (resumen por tabla). 29
10 Endpoints reales de la API PHP. 38
11 Endpoints reales de la API Node. 39
12 Endpoints reales de la API Python. 40
13 Escritura por tabla y consumo por módulo. 41
14 Librerías y dependencias reales. 41
15 Ramas del gateway por dueño y métodos permitidos. 42
16 Ataques comunes y contramedidas del sistema. 58
17 Pruebas de aceptación automatizadas (extracto). 63
18 Variables de entorno por pieza. 68
19 Comparación de enfoques móviles. 78

Índice de figuras

1 Arquitectura de microservicios: cliente web, gateway, tres APIs y base única. 14
2 Casos de uso del sitio público (actor visitante). 16
3 Casos de uso del panel privado (roles admin/operador). 17
4 Clases del dominio: herencia, métodos y asociaciones. 18
5 Clases de servicio y sus dependencias hacia el dominio. 19
6 Componentes y carpetas reales del proyecto. 20
7 Modelo entidad-relación (estilo Chen): entidades, relaciones y atributos. 25
8 Modelo relacional completo: 13 tablas, llaves foráneas internas y referencias lógicas entre dominios. 26
9 Respuesta real del gateway en el navegador: rama MAQUINARIAS con código 200. 37

10 Sitio público recién cargado (vista de escritorio). 44
11 Catálogo público con filtros, orden y paginación. 45
12 Secuencia de inicio de sesión con captcha y JWT. 47
13 Secuencia de creación de contrato con transacción e idempotencia. 49
14 Secuencia de devolución con inspección y cobro final. 50
15 Máquina de estados de la unidad. 51
16 Ciclo de generación y consumo de notificaciones. 52
17 Respuesta real del gateway en el navegador: token inválido recibe 401. 53
18 Panel de gestión real tras iniciar sesión (dashboard). 70
19 Campana de notificaciones abierta. 71
20 Módulo calendario: contratos del mes por estado (33 contratos sembrados). 72
21 Módulo administrar maquinarias: formulario y tabla con datos vivos. 73
22 Edición de maquinaria: el formulario se llena con los datos cargados desde la API Python. . 73
23 Módulo solicitudes: filtro por estado y acciones de aprobar o rechazar. 74
24 Módulo devoluciones: registro con acta de inspección del contrato. 74
25 Sitio público del sistema visto en un celular. 81
26 Login con captcha icónico en el celular. 82
27 Panel privado en el celular con datos vivos. 83
28 Calendario de contratos en el celular. 84
29 Prototipo Expo corriendo de verdad: catálogo con datos vivos del gateway. 91
30 Pestaña de notificaciones del prototipo: el 401 del gateway se gestiona con el mensaje propio de la app. 92

1 Generalidades del sistema

El presente documento describe, con fidelidad al código entregado, el **Sistema de Control de Alquiler de Maquinaria de Construcción** en su integración final: un único punto de entrada (API Gateway) que enruta hacia tres servicios de dominio escritos en PHP, Node.js y Python, un cliente web construido con JavaScript ES6+ y Bootstrap 5, y una base de datos MariaDB con trece tablas controladas por restricciones. El capítulo establece el contexto del proyecto, sus objetivos y el marco conceptual de referencia que sostiene las decisiones técnicas; además recoge, punto por punto, el tratamiento dado a la retroalimentación del Momento 2, porque cada observación recibida se convirtió aquí en cambio verificable en código, base de datos, pruebas o documentación. La regla editorial aplicada a todo el documento es doble: cada punto y subpunto ocupa su propia página para facilitar la revisión, y ninguna afirmación se sostiene sin evidencia (captura, prueba automatizada, script o diagrama regenerado a partir del código real). la regla editorial es doble y se explica aquí para que quien revise sepa qué esperar: cada punto y subpunto ocupa su propia hoja para que ninguna figura ni tabla comparta espacio con otra sección, y cada afirmación fuerte del texto lleva su respaldo inmediato en una captura, un comando, una prueba automatizada o una figura vectorial regenerada a partir del código. Como muestra de esa política, el negocio quedó dividido en maquinarias pesadas y livianas con tablas de subtipo separadas, la entidad Unidad individual hace que cada alquiler bloquee una unidad concreta con serial, y las fechas de la API Node se leen como texto AAAA-MM-DD para que ningún desfase horario corra las cifras del negocio.

1.1 Introducción

La página web entregada en este Momento 3 es un **sistema multiuso de alquiler de maquinaria de construcción** con dos frentes complementarios. El primero es un sitio público (escrito en html, CSS y JavaScript ES6+) donde cualquier visitante consulta el catálogo con fotos, tarifas por día, disponibilidad en tiempo real y envía solicitudes de alquiler que quedan en una cola de gestión. El segundo es un **panel privado** donde administradores y operadores manejan clientes, maquinarias con unidades individuales, contratos de alquiler, devoluciones con acta de inspección, reportes con métricas gráficas, un calendario mensual de contratos y notificaciones del equipo. La página también muestra, con datos vivos de la base, cifras del negocio en el inicio (máquinas, unidades, disponibles y alquileres activos) y permite imprimirlas como comprobante.

Técnicamente todo se articula sobre **una arquitectura de microservicios**: un api **Gateway** en Node.js con Express es el punto único de entrada que recibe cada petición del cliente (roles incluidos), valida un token jwt de quince minutos, aplica límites de tasa y **enruta cada petición a la API dueña** de la tabla implicada: una api en PHP con MySQLi, otra en Node.js con Express y una tercera en Python con FastAPI. La elección de Express como base del gateway responde a su madurez documentada para iniciar servicios web con enrutamiento y middleware (Express.js Project, 2010–2026), y la separación en dominios sigue el patrón API Gateway descrito por Richardson (2019–2026). La interfaz web usa JavaScript vanilla ES6+ y Bootstrap 5, tal como exige el enunciado del entregable.

1.2 Objetivo general y específicos

Objetivo general. Entregar una aplicación web integral —con documentación correspondiente al código real— que gestione el alquiler de maquinaria de construcción pública y privadamente con un patrón API Gateway, tres microservicios dueños de tablas, una base única normalizada y evidencia automatizada de funcionamiento y seguridad.

Objetivos específicos.

1. Publicar el catálogo con fotos, tarifa diaria, categoría, disponibilidad por unidad y paginación, consumible sin sesión.
2. Registrar clientes con documento único y contacto válido (teléfono o correo).
3. Crear alquileres sobre unidades individuales con cálculo de días, tarifa pactada e idempotencia de contratos.
4. Gestionar devoluciones con acta de inspección, cobro de retraso a *tarifa pactada* y estado final de la unidad.
5. Aprobar o rechazar solicitudes públicas; al aprobar, crear el contrato dentro de una transacción.
6. Entregar métricas gráficas vivas (dashboard), calendario mensual de contratos y notificaciones del equipo.
7. Controlar el acceso por roles con captcha obligatorio, bloqueo por intentos y token de sesión en memoria.
8. Proteger el sistema de ataques web habituales (SQLi, XSS, CSRF, DoS, *path traversal*, *rebinding* de DNS y otros), con evidencia de bloqueo.
9. Automatizar la puesta en marcha y la siembra idempotente de datos de demostración para demostrar el sistema completo en cualquier equipo académico.

1.3 Marco teórico

El proyecto se apoya en cuatro conceptos técnicos. Primero, la arquitectura de microservicios divide el sistema en servicios independientes que se comunican por HTTP y JSON, cada uno responsable de su propio dominio de datos (IBM Cloud Education, 2023a). Segundo, el estilo REST define los recursos y los métodos de la web aplicada al negocio (Mozilla Foundation, 2024b). Tercero, las bases de datos relacionales normalizadas evitan la redundancia y las anomalías, con llaves primarias, foráneas y CHECK para la integridad (MariaDB Foundation, 2024). Cuarto, PHP, Python y Node.js aportan los tres lenguajes de servidor del proyecto (Grupo PHP, 2024; Python Software Foundation, 2024; FastAPI, 2024), cada uno con su documentación oficial en español. Esta base conceptual justifica las decisiones de diseño que se detallan en las secciones siguientes.

1.4 Correcciones aplicadas de la retroalimentación del Momento 2

La retroalimentación del docente en el Momento 2 fue la hoja de ruta de esta versión. La Tabla 1 lista cada observación recibida, la corrección aplicada y dónde quedó la evidencia dentro del documento y del código. Ninguna observación quedó sin atender: las que afectaban el modelo se convirtieron en cambios de esquema y API, las de diagramas se resolvieron reescribiendo los fuentes PlantUML y las de documentación se abordaron integrando el material faltante en el cuerpo de esta entrega.

Tabla 1

Observaciones del Momento 2 y su corrección en la versión final.

Observación (Momento 2)	**Corrección aplicada (con evidencia)**
Falta tabla de unidades individuales; el alquiler no apunta a una unidad concreta	Tabla `unidades` con `idunidad, idmaquinaria, serial UNIQUE, estado ENUM`; `alquileres.idunidad` reserva la unidad real. Evidencia: anexo de esquema (cap. 4) y el modelo relacional 8.
Reserva sobre contador genérico y doble venta de la unidad ocupada	`SELECT…FOR UPDATE` dentro de transacción; unidad ocupada responde 409. Evidencia: 13 y prueba E2E del cap. 8.
El cobro de retraso usaba la tarifa actual, no la del contrato	`tarifa_pactada` se guarda en el alquiler; el retraso y el total se calculan con ella. Evidencia: 14 y reglas de cálculo del cap. 6.
Sin CHECK ni ENUM (estados en texto libre)	ENUM en todos los estados y CHECK de fechas/montos/contactos en `alquileres`, `solicitudes`, `clientes`, `unidades`, `reportes`. Evidencia: esquema del cap. 4.
Borrado físico de maquinarias (el historial mostraba eliminar con contratos activos)	Borrado lógico `activo=0` con bloqueo de desactivación si hay unidades ocupadas. Evidencia: API Python cap. 5 y diccionario cap. 4.
`idmaquinaria` redundante en `inspecciones`	Eliminado por 3FN: se deduce de `idalquiler` → `alquileres.idunidad`. Evidencia: Tabla 7.
`disponibles` guardado y contaminado en dos lugares	Se computa con `COUNT` al leer; nunca se escribe un contador. Evidencia: cap. 5 (API Python) y JSON del catálogo cap. 6.
Alquiler_inspección con cardinalidades inconsistentes	Normalizada a ”1 a 0..1” con `UNIQUE idalquiler` y FK interna del dominio Node. Evidencia: 4 y 8.
FK entre bases inexistentes sin documentación	Documentadas como *referencias lógicas* con mecanismos compensatorios (Tabla 8) y el modelo relacional completo (8).
Clientes con teléfono y correo ambos obligatorios	CHECK permite contacto con teléfono o correo (uno válido basta). Evidencia: esquema cap. 4 y prueba de registro cap. 8.
Diagrama de clases copiado del DER (PK/FK, sin tipos, sin métodos)	Reescrito como diagrama OO: asociaciones, tipos, `Maquinaria` abstracta y métodos reales. Evidencia: 4–5.
Clases de servicio sueltas sin dependencias al dominio	Diagrama d03b une gateway y servicios con flechas a sus clases de dominio. Evidencia: 5.
Falta ”Consultar catálogo” y ”Gestionar maquinaria” en casos de uso	Incluidos como UC1 y UC6; además `<<include>>` en devolución/inspección y login/captcha. Evidencia: 2–3.
`fechalquiler` marcada como PK en Alquiler	Corregido: PK `idalquiler` y atributos de fecha sin marcas. Evidencia: 4.

Observación (Momento 2)	**Corrección aplicada (con evidencia)**
`PUT disponibles` por contador apuesta a carreras	Reemplazo por el flujo transaccional con bloqueo de fila; los verbos REST por recurso. Evidencia: 13 y cap. 5.
POST responde 200 en creación; sin stock respondía 403	Creaciones responden 201; sin disponibilidad responde 409. Evidencia: Tabla de HTTP RNF01 y pruebas cap. 8.
Zona horaria con desfase UTC en fechas	`dateStrings:true` en el pool Node y fechas `AAA-MM-DD`. Evidencia: cap. 5 y JSON cap. 6.
API PHP no era REST (rutas scripts con parámetros)	REST por recurso: `/usuarios`, `/clientes[/id]`, `/captcha`, `/bitacora`. Evidencia: cap. 5 y Tabla 15.
Python escribía en la base de otra API sin aclaración	Documentada la escritura exclusiva: Python solo toca `maquinarias*`, `unidades` y `reportes`. Evidencia: Tabla de dueños cap. 5.
Faltaban tildes; tablas como imágenes de baja resolución	Documento 100 % texto vectorial PDF con Times New Roman: todas las tablas son texto y las figuras son PDF vectorial legible.
El script SQL capturado quedaba incompleto	Se entrega el script completo en el cuerpo del cap. 4, sin comentarios ni semillas.
Referencias no citadas y fuera de tema	Bibliografía técnica citada en el texto (APA 7) y disponible al final (cap. de Referencias).
Numeración desordenada y secciones vacías	Estructura de 13 puntos y subpuntos: cada subpunto incluye página nueva y texto argumentado; las secciones vacías de la entrega anterior fueron llenadas o eliminadas.
Captcha y bloqueo sin especificar códigos	La tabla del cap. 7 separa 400 (captcha), 401/403 (credenciales/cuenta) y 429 (IP). Evidencia: 12 y pruebas cap. 8.

2 Definición del problema y requisitos

Este capítulo traduce el problema n. 15 (control del alquiler de maquinaria de construcción) a requisitos verificables. El enunciado académico plantea dos escenarios críticos que el diseño debe resolver: el cobro correcto de una devolución tardía o con perjuicios, y la reserva que solo se firma si existe una unidad concreta disponible. La respuesta arquitectónica fue modelar *unidades individuales* con serial y estado y apuntar cada contrato a una de ellas, además de conservar la tarifa pactada como hecho histórico del contrato. A partir de esos dos ejes se derivan los treinta requisitos funcionales y los catorce no funcionales que el código implementa, listados a continuación con su trazabilidad a módulo, prueba y evidencia. cada requisito de la tabla responde a una regla del código verificable: RF07 y RF08 por el bloqueo `FOR UPDATE` y la clave de idempotencia, RF09 por el cálculo de retraso con la tarifa pactada (que esta versión corrigió para no cobrar el día de atraso dos veces), RF14 por el captcha de un solo uso con cuota por IP, RF21 por las notificaciones automáticas y RF26 por la siembra idempotente de datos de demostración. Los no funcionales fijan números concretos en vez de promesas: puertos fijos en loopback, códigos HTTP exactos, gzip del 74 por ciento en CSS, bcrypt con costo 12, purga de bitácora a los 180 días y tope de cuerpo de 1 MB en el gateway.

2.1 Planteamiento del problema

El proyecto atiende el problema n.° 15: el control del alquiler de maquinaria de construcción. Los contratos establecen fechas de entrega y devolución para calcular el costo por días. Al recibir el equipo se levanta un acta de inspección con estado físico y desgaste. Hoy ese control vive en papeles: no se sabe qué unidad concreta salió, el retraso se cobra "a tarifa del día" y los equipos se borran con contratos activos encimados.

La rúbrica del problema exige dos escenarios límite que el sistema debe cubrir con su diseño: (1) la **devolución tardía o con daños**, que genera cobros automáticos de retraso y reparación; y (2) la **reserva sin disponibilidad real**, que impide firmar el contrato si la unidad concreta no está lista (por ejemplo, si todavía no diligencia la inspección del alquiler anterior). Para el segundo escenario el modelo guarda *unidades individuales* con serial y estado (idunidad, idmaquinaria, serial, estado: disponible / alquilada / en_inspección / en_reparación) y el alquiler apunta a `idunidad`, que es exactamente la observación principal de la retroalimentación del Momento 2 sobre el esquema anterior.

Ambos escenarios están implementados y probados con la suite E2E de la sección 8: la devolución tardía cobra los días de atraso multiplicados por la *tarifa pactada* y suma la reparación; una unidad ocupada responde 409 y un salto de estado prohibido responde 400.

2.2 Alcance del sistema

El sistema incluye el sitio público (catálogo con fotos y filtros, beneficios, formulario de solicitud con costo estimado y contacto), el panel privado con doce módulos, notificaciones del equipo, calendario mensual de contratos, gráficas vivas, impresión de módulos, respaldos diarios con retención y una base de datos única con tubería de usuarios de derecho. *No incluye*: pagos en línea, facturación electrónica, despliegue en la nube, autenticación con proveedores externos, ni aplicación móvil nativa (el análisis teórico de móvil y un prototipo real se entregan en la sección 10 y el anexo D).

2.3 Requisitos funcionales

La Tabla 2 lista los requisitos funcionales completos del código entregado (también incluye los del documento guía vigente, en versión ajustada al gateway).

Tabla 2

Requisitos funcionales completos del código entregado.

Cód.	Requisito funcional
Cód.	**Requisito funcional**
RF01	Registrar clientes con nombre, documento único y contacto válido (teléfono o correo, obligatorio uno).
RF02	Consultar el catálogo público con fotos reales, tarifa diaria, categoría y unidades totales/disponibles calculadas al leer.
RF03	Filtrar por categoría (pesada/liviana), disponibilidad, rango de tarifa y orden (5 criterios), con paginación de 9 en 9.
RF04	Consultar el catálogo interno con el mismo motor de filtros para el equipo.
RF05	Gestionar maquinaria: crear con foto (JPG/PNG real hasta 2 MB), editar tarifa y costo de movilización, agregar unidades, desactivar (borrado lógico) y reactivar.
RF06	Gestionar unidades por máquina: crear en lote con serial único al crear la máquina, agregar más después, cambiar estado manual con reglas.
RF07	Crear alquileres sobre una unidad concreta con fecha, tarifa pactada, movilización y cálculo automático de días.
RF08	Impedir contratos sobre unidades ocupadas (409) e impedir duplicados con llave de idempotencia (200 con el mismo contrato).
RF09	Registrar la devolución con fecha real, estado físico, nivel de desgaste, daños y costo de reparación; calcular retraso y total al leer con la tarifa pactada.
RF10	Generar la acta de inspección (única por contrato) y sugerir el estado final de la unidad (disponible / en inspección / en reparación).
RF11	Mostrar el historial de contratos con la unidad asignada, total y estado.
RF12	Recibir solicitudes públicas con contacto y fechas, con cuota por IP y trampa anti—espam (honeypot).
RF13	Gestionar solicitudes: filtrar por estado y aprobarlas (crea el contrato en transacción) o rechazarlas con motivo.
RF14	Iniciar sesión con captcha obligatorio de un solo uso (cuota por IP); bloquear la cuenta tras 5 fallos (15 min) y limitar por IP.
RF15	Cambiar la contraseña propia con la clave actual (mínimo 12 caracteres) y restablecer la de otros usuarios (rol administrador).
RF16	Gestionar cuentas de usuario: listar, crear con clave temporal que obliga a cambiar al primer ingreso (solo administrador).
RF17	Controlar acceso por roles: administrador (todo), operador (21 acciones exactas en la matriz del gateway) y visitante sin cuenta solo en el sitio público.
RF18	Mostrar el resumen del negocio con 4 métricas y 4 gráficas de datos vivos.
RF19	Generar y guardar reportes del resumen (listar y borrar solo administrador).
RF20	Ver un calendario mensual de contratos con filtros por estado.
RF21	Notificar eventos del equipo (contratos, devoluciones, solicitudes) en una campana con contador, listar y marcar leídas.
RF22	Imprimir el módulo activo (impresión limpia sin menús ni botones).
RF23	Respaldar la base diariamente con mysqldump (retención de 7) y purgar automáticamente la bitácora antigua (180 días).
RF24	Verificar la salud de las cinco piezas con `/salud` y 56 comprobaciones automáticas del proyecto.
RF25	Registrar toda operación de escritura en una bitácora interna con ip/usuario/código.
RF26	Sembrar datos de demostración idempotentes y registrar máquinas nuevas por carpetas del frontend.

RF27	Validación en vivo en formularios (nombre, documento, teléfono/correo, fechas reales y rangos de dinero).
RF28	Doble confirmación en acciones destructivas y botones con bloqueo de doble clic.
RF29	Notificar el cambio de tokens (sesión expirada) al recargar al formulario de acceso.
RF30	Prevenir ataques web comunes (tabla 16 de la sección 7) incluyendo HTTPS, cifrado, captcha, límites de tasa y varios encabezados de seguridad.

2.4 Requisitos no funcionales

La Tabla 3 completa los atributos de calidad exigidos por el enunciado y ampliados con los del código entregado.

Tabla 3

Requisitos no funcionales completos.

Cód.	Requisito no funcional
Cód.	**Requisito no funcional**
RNF01	Responder con códigos HTTP correctos: 200, 201, 400, 401, 403, 404, 405, 409, 413, 421, 429 y 500.
RNF02	Usar puertos fijos en 127.0.0.1: 8443 (front), 3000 (gateway), 8000 (PHP), 3002 (Node), 3003 (Python), 3306 (MariaDB).
RNF03	Base única controlalquiler con credenciales e identidad solo por variables de entorno y usuarios de base con mínimo privilegio.
RNF04	Entregar las fechas como AAAA-MM-DD sin desplazamiento horario (dateStrings).
RNF05	Estilo REST consistente (rutas por recurso y métodos correctos en las cuatro piezas).
RNF06	Seguridad en capas: TLS 1.2, CSP, HSTS, nosniff, COOP/CORP, SRI, captcha, bcrypt 12, bloqueo, límites de tasa, verificación de host y DNS, idempotencia e integridad SHA-256.
RNF07	Rendimiento: gzip en textos con ahorro del 74% en CSS, ETag con revalidación 304, timeouts de conexión y max_statement_time en BD.
RNF08	Observabilidad: bitácora interna, registros por servicio, `/salud` en la 5 piezas y scripts de verificación.
RNF09	Usabilidad: modo oscuro automático, radios suaves, objetivos de 44 px, impresión limpia y avisos con toasts (nunca alert).
RNF10	Accesibilidad: skip-link, ARIA en filtros y robots adecuados, contraste verificado, foco visible.
RNF11	Compatibilidad: navegadores modernos (Chrome, Edge, Firefox) y responsive (escritorio, tableta y celular).
RNF12	Mantenimiento: código por módulos, responsabilidades separadas y nombres de dominio en español.
RNF13	Reproducibilidad: un solo comando de arranque con datos semilla idempotentes y 26 pruebas E2E en verde.
RNF14	Resistencia: transacciones con bloqueo de filas, idempotencia para eliminar duplicados y borrado lógico cuando aplique.

3 Arquitectura y diseño del sistema

La arquitectura implementada es un patrón *API Gateway* con tres servicios de dominio que comparten una base de datos relacional pero no se llaman entre sí: el único canal de comunicación es `HTTPS` y el formato de mensajes es uniforme `{TOKEN, dat}`. Este capítulo justifica por qué la centralización simplifica la interfaz del cliente (un solo origen, un solo formato, una sola política de seguridad), cómo queda repartida la escritura de tablas entre PHP, Node.js y Python, y qué mecanismos compensan la decisión de no duplicar rutas: validación previa, transacciones con bloqueo de fila, idempotencia y una matriz cerrada de permisos por rol. Los diagramas de arquitectura, casos de uso, clases y componentes fueron regenerados para esta entrega a partir del código vigente. dos secciones nuevas cierran el capítulo: la matriz de quién puede hacer qué (visitante, operador y administrador, con el rol de solo lectura ya eliminado del código) y el inventario completo de los doce módulos del panel más la campana global, con la API dueña que alimenta a cada uno. Así un evaluador puede verificar la arquitectura sin abrir el código: el diagrama de componentes dice dónde viven las piezas, la matriz de permisos dice quién pasa por cada puerta, la tabla de módulos dice qué consume cada pantalla, y el contrato de ramas (capítulo 5) dice por dónde entra cada petición al gateway.

3.1 Arquitectura de microservicios

El sistema se organiza en **microservicios**: procesos independientes que se comunican por HTTP/JSON, cada uno dueño exclusivo de sus tablas, con despliegue y ciclo de vida separados. La Figura 1 muestra el conjunto: un **API Gateway** (Node/Express, puerto 3000) concentra la entrada, aplica su pipeline de doce filtros y enruta a tres APIs de dominio (PHP 8000, Node 3002, Python 3003) que escriben sobre la base única `controlalquiler`. El cliente web se sirve con su propio servidor estático (puerto 8443, HTTPS) y *solo* dialoga con el gateway.

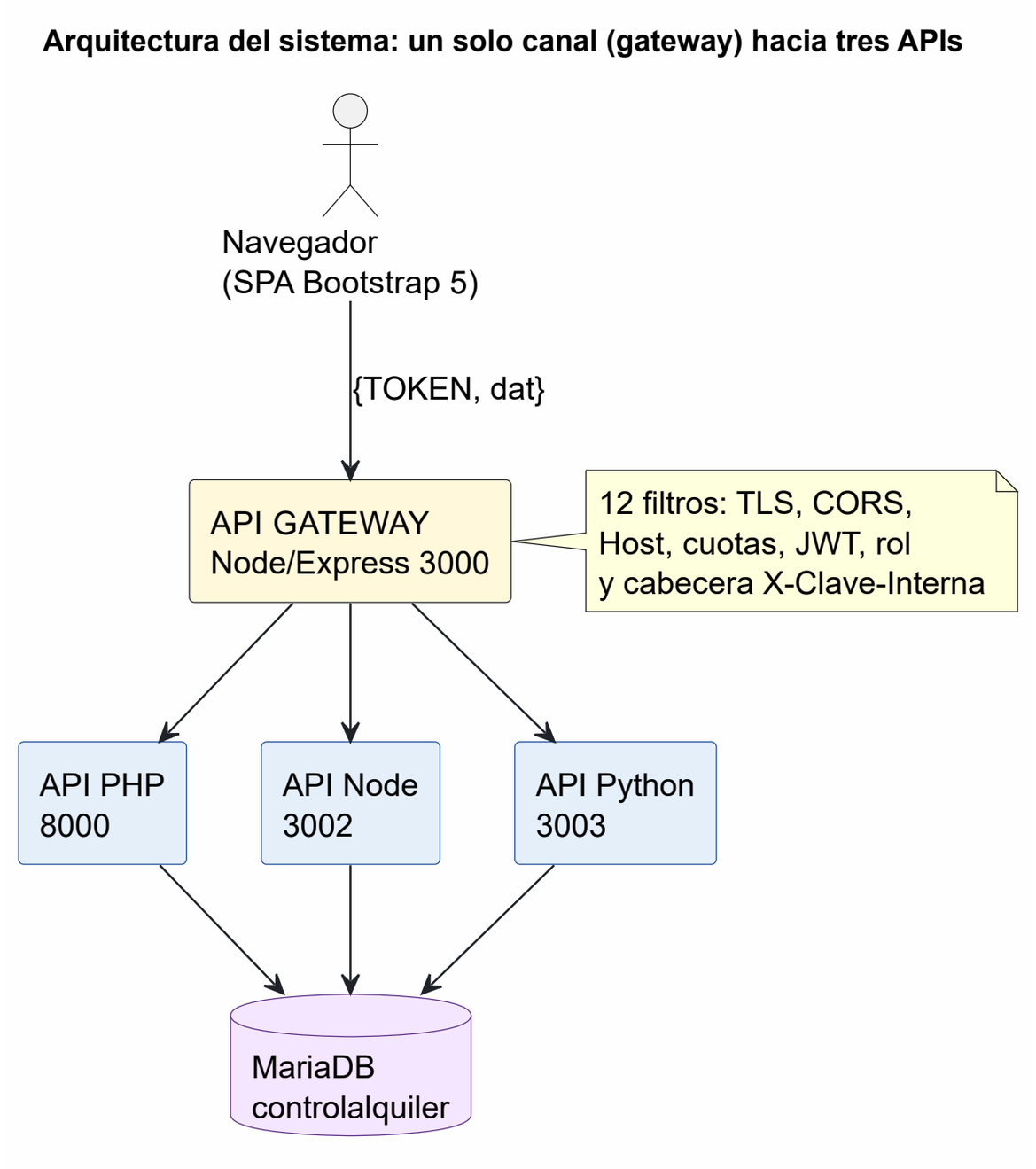


Figura 1

Arquitectura de microservicios: cliente web, gateway, tres APIs y base única.

Tabla 4

Puertos y responsables de cada pieza.

Pieza	Puerto	Tablas que escribe	Responsabilidad
API Gateway	3000	ninguna	Autenticación JWT, permisos, límites, ruteo y bitácora.
API PHP	8000	clientes, usuarios, captchas, bitácora, (lee nada ajeno)	Identidad, roles, cuentas, captcha y registro de eventos.
API Node	3002	alquileres, inspecciones, solicitudes, notificaciones	Contratos con transacción, devoluciones, solicitudes y avisos.
API Python	3003	maquinarias, maquinariaspesadas, maquinariaslivianas, unidades, reportesguardados	Catálogo, unidades, fotos en disco y reportes.
Servidor estático	8443	ninguna (solo lee su carpeta)	HTTPS, cabeceras de seguridad, integridad y caché del SPA.

3.2 Diagrama de casos de uso

Las Figuras 2 y 3 presentan los quince casos de uso con los cuatro actores del sistema, divididos en dos diagramas para poder leerlos con claridad: el sitio público del visitante y el panel privado del equipo. Incluyen las relaciones `<<include>>` exigidas por la retroalimentación (la devolución *incluye* el levantar el acta de inspección; generar el resumen incluye su cálculo; aprobar una solicitud incluye crear el contrato; e iniciar sesión incluye resolver el captcha), además de los casos que faltaban en la entrega anterior (consultar catálogo y gestionar maquinaria).

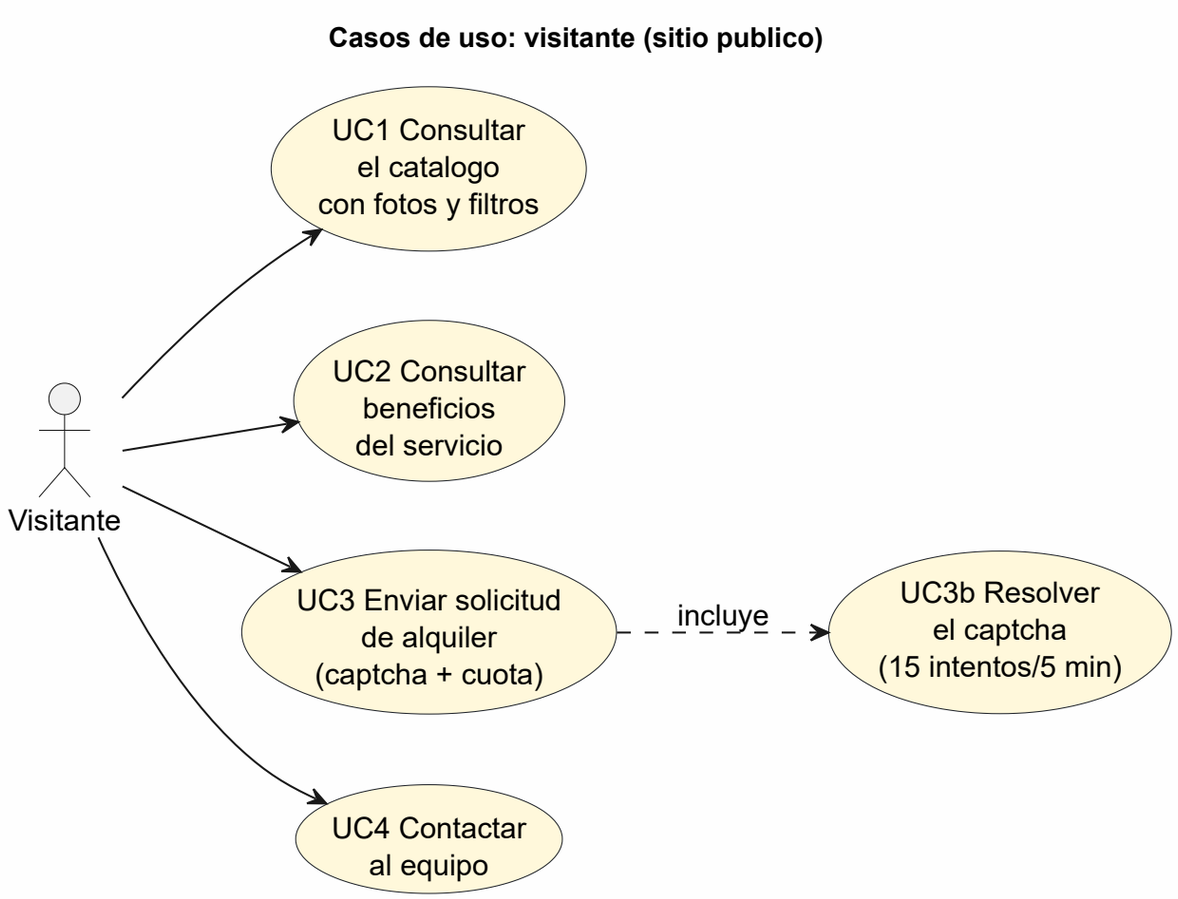


Figura 2

Casos de uso del sitio público (actor visitante).

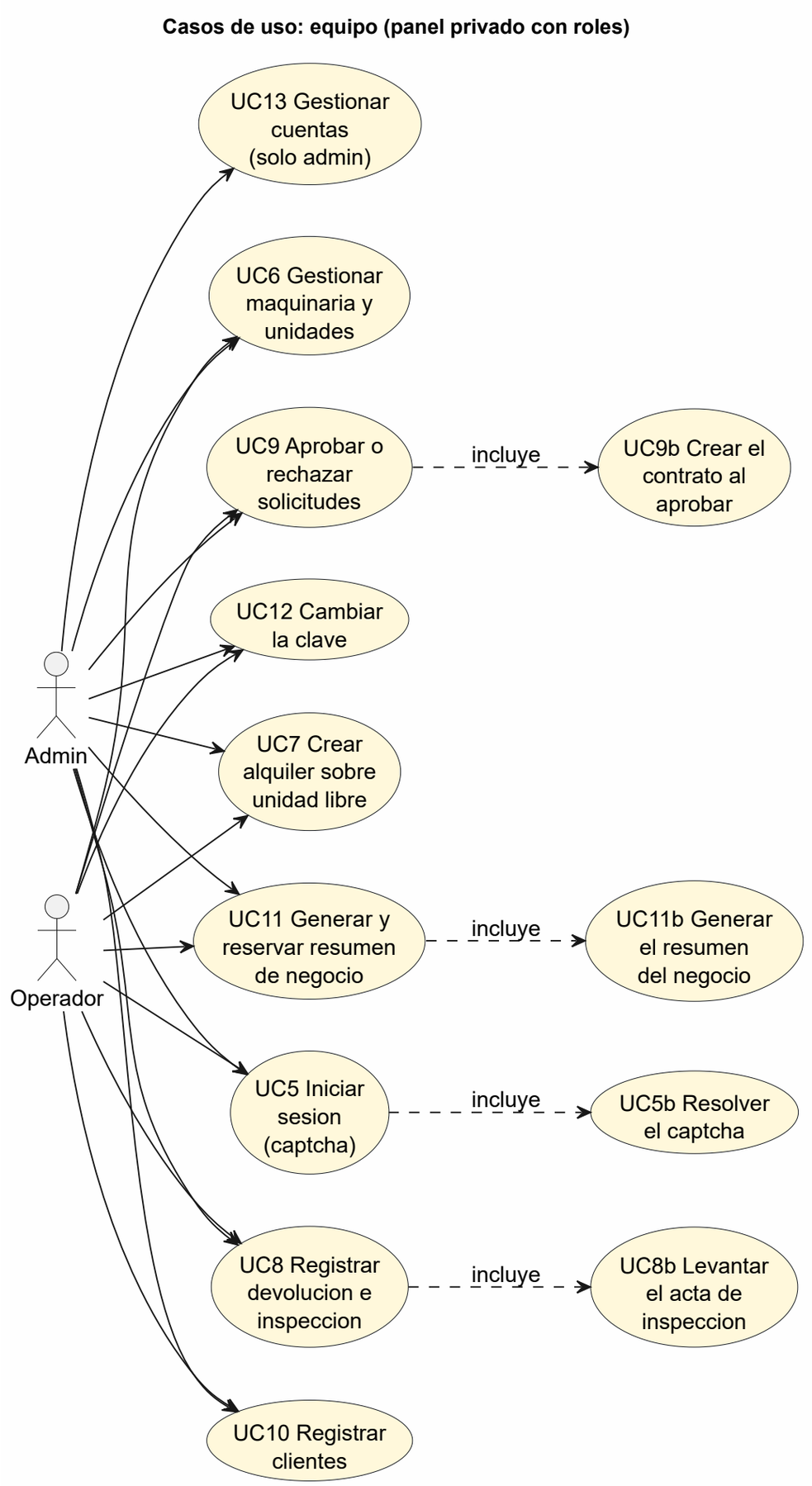


Figura 3

Casos de uso del panel privado (roles admin/operador).

3.3 Diagrama de clases del dominio

Las Figuras 4 y 5 corrigen el diagrama del Momento 2 siguiendo la retroalimentación del docente: clases orientadas a objetos (asociaciones entre objetos, sin marcas de PK/FK, que pertenecen a la base de datos), atributos con tipos de dato, `Maquinaria` **abstracta** con la herencia Pesada/Liviana, métodos de negocio realmente implementados en las APIs (`reservarUnidad()`, `liberarUnidad()`, `calcularRetraso()`, `calcularReparacion()`) y, por último, las clases de servicio unidas a su dominio con dependencias visibles. En el diagrama de clases la categoría de la maquinaria la expresa la herencia; en el esquema SQL la columna ENUM `categoria` se conserva deliberadamente como discriminador del subtipo para las vistas de catálogo. La cardinalidad Alquiler—Inspección queda `1 --- 0..1`, coherente con la llave `UNIQUE` de la base.

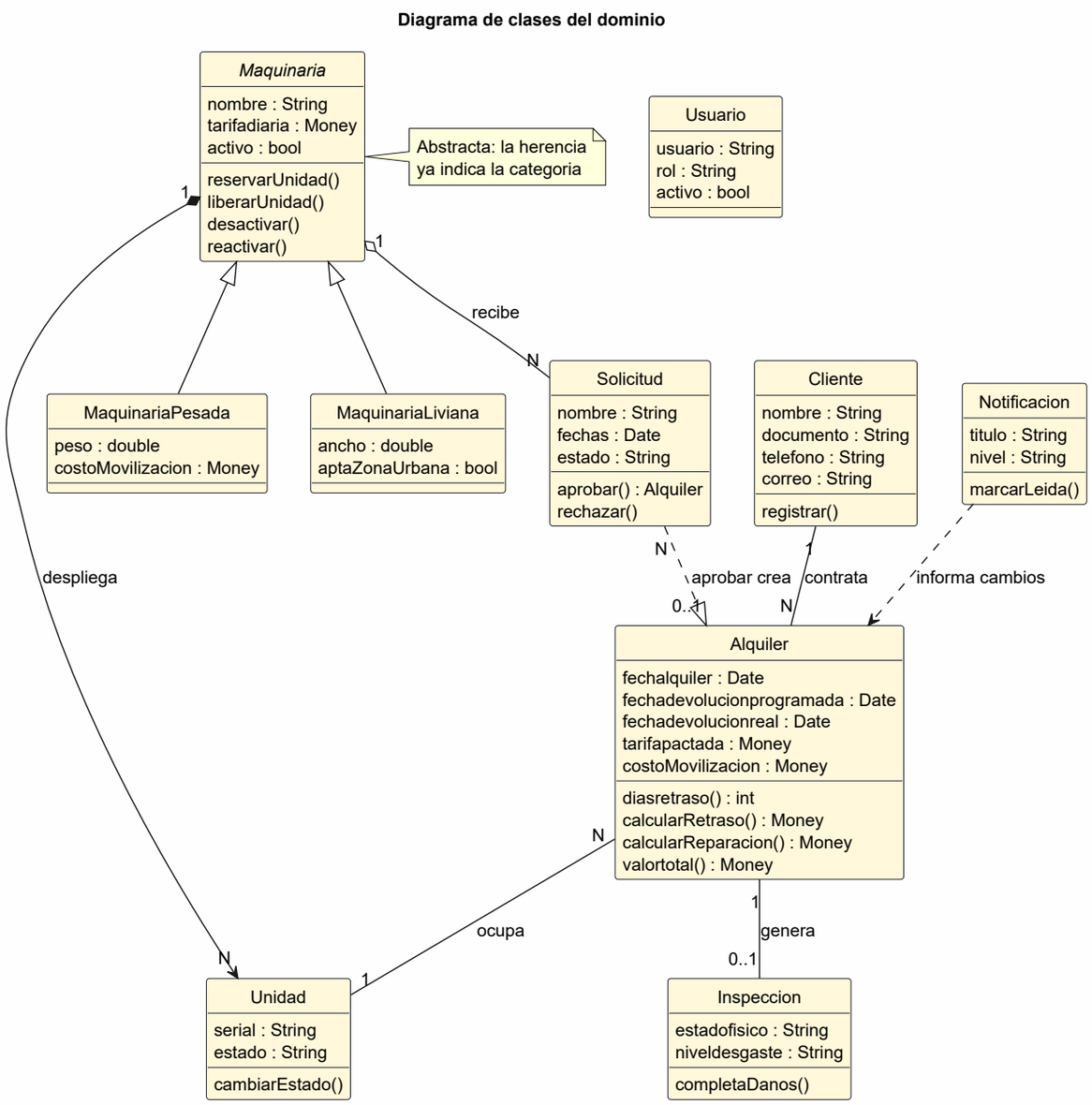


Figura 4

Clases del dominio: herencia, métodos y asociaciones.

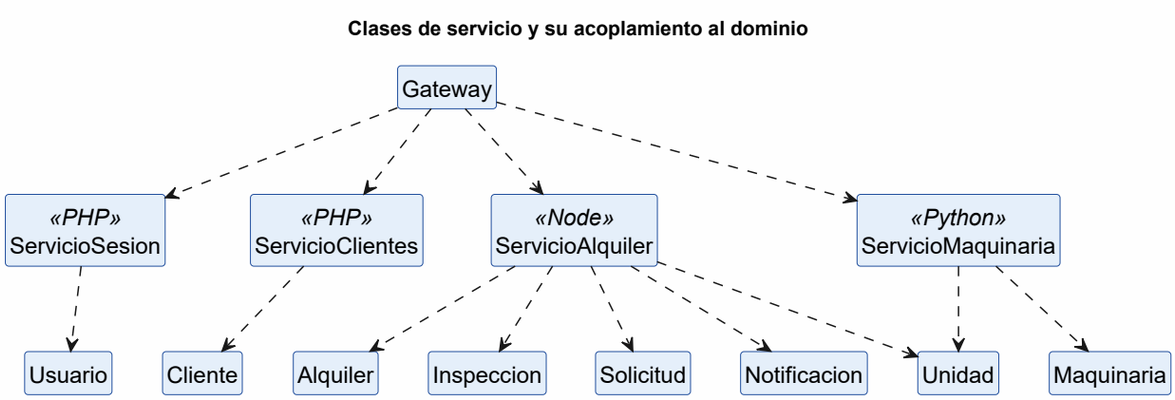


Figura 5

Clases de servicio y sus dependencias hacia el dominio.

3.4 Diagrama de componentes y estructura del código

La Figura 6 mapea los componentes a las carpetas reales del repositorio. El patrón aplicado es **front—controller con pipeline de filtros** en el gateway (cada petición atraviesa: acceso por IP/origen, límites por rama, validación de estructura, candado anti-*brute-force*, cuota de fotos/captcha/solicitudes, autenticación JWT, autorización por rol y ruteo); en las APIs de dominio se usa **REST** por recurso con repositorios de BD (`mysqli`, pool `mysql2/promise`, pool `mysql-connector`); y la consistencia de negocio se obtiene con **transacciones ACID locales** (`BEGIN`/`COMMIT`/ `ROLLBACK`): al existir una única base de datos no se requiere un patrón Saga distribuido, pues si un paso falla el `ROLLBACK` no deja registros parciales.

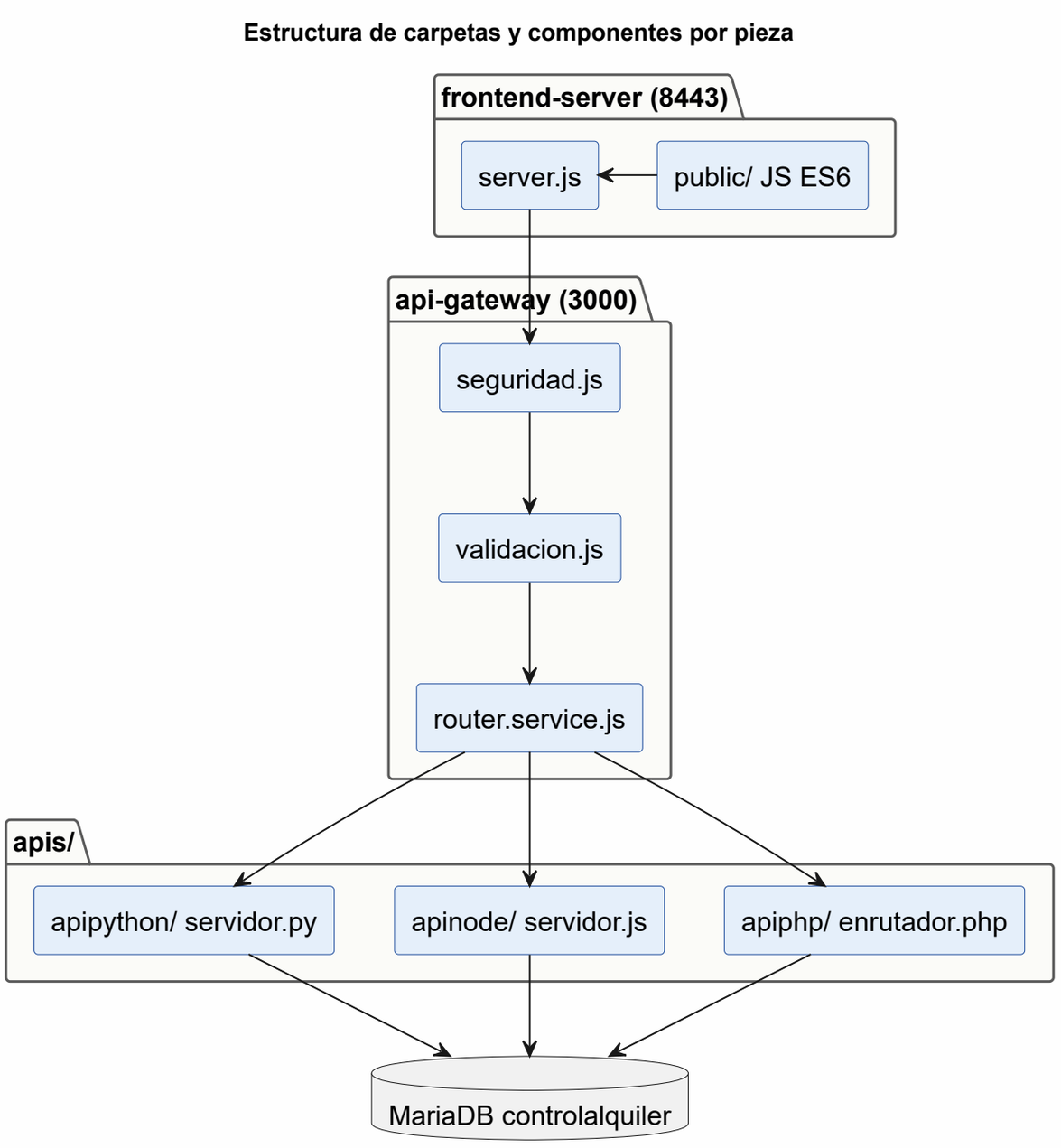


Figura 6

Componentes y carpetas reales del proyecto.

3.5 Roles del sistema: qué puede y qué no puede cada uno

El sistema reconoce tres figuras: el **visitante** (sin cuenta, navega el sitio público) y dos roles autenticados, **operador** y **administrador**. El rol de *solo lectura* existía en la entrega anterior y fue retirado por completo del código en esta versión: ya no existe en la matriz del gateway, la API PHP solo crea cuentas de operador, el ENUM de la base quedó con dos valores, el frontend ya no lo lista y las pruebas se reescribieron (la cuenta de demostración fue eliminada de la base al aplicar el cambio). La Tabla 5 resume, con un lenguaje plano, qué puede y qué no puede hacer cada figura; el control vive en un único lugar del gateway (el objeto `PERMISOS` de `permisos.js`, rama a rama y método a método) y la API Python exige además exclusivamente el rol de administrador en sus rutas destructivas (cabecera `x-rol`) como segunda verificación.

Tabla 5

Qué puede hacer cada figura del sistema.

Acción	Visitante	Operador	Admin
Ver catálogo público, fotos y precios	Sí (público)	Sí	Sí
Enviar una solicitud de alquiler con captcha	Sí (5 por 15 min)	Sí	Sí
Entrar al panel privado	No (401)	Sí	Sí
Cambiar su propia clave (exige la actual)	—	Sí	Sí
Crear y consultar clientes	—	Sí	Sí
Crear alquiler sobre unidad libre	—	Sí	Sí
Registrar devolución con acta e inspección	—	Sí	Sí
Aprobar o rechazar solicitudes	—	Sí	Sí
Crear o editar maquinaria y subir su foto	—	Sí	Sí
Crear unidades con serial y cambiar estados	—	Sí	Sí
Dar de baja lógica una maquinaria	—	No[1]	Sí
Ver, guardar y borrar reportes del resumen	—	Sí	Sí
Ver y marcar notificaciones de la campana	—	Sí	Sí
Crear cuentas de usuario y restablecer claves ajenas	—	No (403)	Sí
Ver el panel completo (12 módulos + campana)	—	Sí	Sí

Para el operador quedan 21 acciones exactas en la matriz del gateway —cambiar su propia clave; leer y marcar notificaciones; crear y consultar clientes; crear alquileres y registrar devoluciones; gestionar maquinaria (crear, editar, subir foto, baja con doble restricción) y sus unidades; y el ciclo completo de solicitudes y reportes— y la prueba E2E 20 lo verifica recibiendo 403 al intentar gestionar cuentas, acción reservada al administrador, que pasa por dado con el comodín de acceso total.y el administrador pasa todo con el comodín *.

[1] La baja lógica exige administrador dos veces: la matriz del gateway lo permite a operador (`MAQUINARIAS:DELETE`), pero la API Python vuelve a exigir `admin` en `x-rol`; así la última palabra siempre es del dominio.

3.6 Los módulos del panel: cuáles son y qué hace cada uno

El panel privado tiene **doce módulos de menú** (los data-modulo reales del código: `inicio`, `clientes`, `catalogo`, `solicitudes`, `maquinas`, `unidades`, `alquileres`, `calendario`, `devoluciones`, `reportes`, `usuarios` y `clave`) **más una campana global de notificaciones** persistentes en la barra, que no es un módulo de menú sino un panel siempre visible. La Tabla 6 indica, por módulo, qué hace, a qué API dueña habla y quién lo usa tras retirar el rol lemme.

Tabla 6

Módulos del panel, función, dueño de datos y quién lo usa.

Módulo	Función	Dueño (API)	Quién lo usa
Inicio (dashboard)	cifras vivas + 4 gráficas	PHP+Node+Python	admin/operador
Clientes	alta y búsqueda documental	PHP	admin/operador
Catálogo	catálogo interno con filtros y detalle	Python	admin/operador
Maquinas	crear/editar/desactivar maquinaria	Python	admin/operador
Unidades	crear unidades por serial y cambiar estados	Python	admin/operador
Alquileres	contratos con tarifa pactada e idempotencia	Node	admin/operador
Devoluciones	acta de inspección, retraso y totales	Node	admin/operador
Calendario	vista mensual de contratos por color	Node/Python	admin/operador
Solicitudes	aprobar (crea contrato) o rechazar	Node	admin/operador
Reportes	resumen del negocio y guardados	Python	admin/operador
Usuarios	listar cuentas y crear operador	PHP	admin
Clave	cambio de la clave propia	PHP	admin/operador
Campana	notificaciones con badge y toasts	Node	admin/operador

Los identificadores listados son los mismos `data-modulo` que el SPA usa para mostrar u ocultar el menú con el `rol` entregado por el login: `usuarios` solo aparece a administrador, `solicitudes` a los dos roles y el resto de módulos pide sesión de cualquiera de los dos. Las impresiones del módulo activo y el modo oscuro automático acompañan a todos por igual.

4 Modelado y diseño de la base de datos

La base única `controlalquiler` es el núcleo de la coherencia del sistema: toda regla de negocio crítica (colisión de reservas con unidades ocupadas, tarifa pactada como hecho histórico, acta de inspección única por contrato) termina siendo una restricción declarada en SQL, una transacción o una validación de API. Este capítulo documenta el modelo conceptual estilo Chen, el DER físico dividido por dueño para reflejar la separación lógica de dominios, la aplicación de las tres formas normales con su respuesta explícita a la retroalimentación del docente, el diccionario de datos alineado con el SQL real y el script de creación completo incrustado en el cuerpo, sin comentarios ni datos semilla, tal como se solicitó. el modelo relacional completo (13 tablas en una misma figura horizontal) hace visible de un vistazo las dos familias de llaves: las foráneas reales del esquema (`inspecciones.idalquiler` con RESTRICT, los subtipos de maquinaria en cascada y `unidades.idmaquinaria` con restricción) y las referencias lógicas entre dominios, dibujadas con líneas discontinuas porque su validez la comprueba el gateway con transacciones y no la base con integridad referencial. El script final del capítulo se puede ejecutar tal cual: solo creación de tablas, sin semillas ni comentarios, idéntico al que la talla del arranque aplica en modo compartido. La base de datos es el corazón del sistema y de la separación de responsabilidades por dominio. Este capítulo documenta su diseño de extremo a extremo: primero el modelado conceptual con el diagrama entidad—relación estilo Chen, luego el modelo relacional completo de las trece tablas, la justificación de la normalización, el diccionario de datos que coincide letra por letra con el script real, y por último el script de creación completo, incrustado aquí mismo y no en un anexo, para que quien revise compare directamente lo descrito con lo ejecutado. Todas las tablas viven en una única base `controlalquiler` (MariaDB, motor InnoDB, charset `utf8mb4_unicode_ci`) y la decisión de separar la escritura por dominio se justifica abajo con sus compensaciones verificables.

4.1 Modelo entidad—relación y modelo relacional completo

Las dos figuras grandes de esta sección muestran el modelado. La Figura 7 presenta el modelo conceptual estilo Chen (entidades en rectángulos, relaciones en rombos y atributos en óvalos) y la Figura 8 despliega el modelo relacional completo: las trece tablas juntas, sus llaves primarias, las llaves foráneas reales del esquema (líneas continuas) y las referencias lógicas entre dominios (líneas discontinuas, validadas por HTTP y por transacción). El cambio estructural clave frente al Momento 2 fue crear la entidad **Unidad** (`idunidad, idmaquinaria, serial UNIQUE, estado ENUM`) y hacer que cada alquiler apunte a `idunidad`: la reserva bloquea hoy una unidad concreta y no un contador, tal como observó la retroalimentación (la observación más importante recibida).

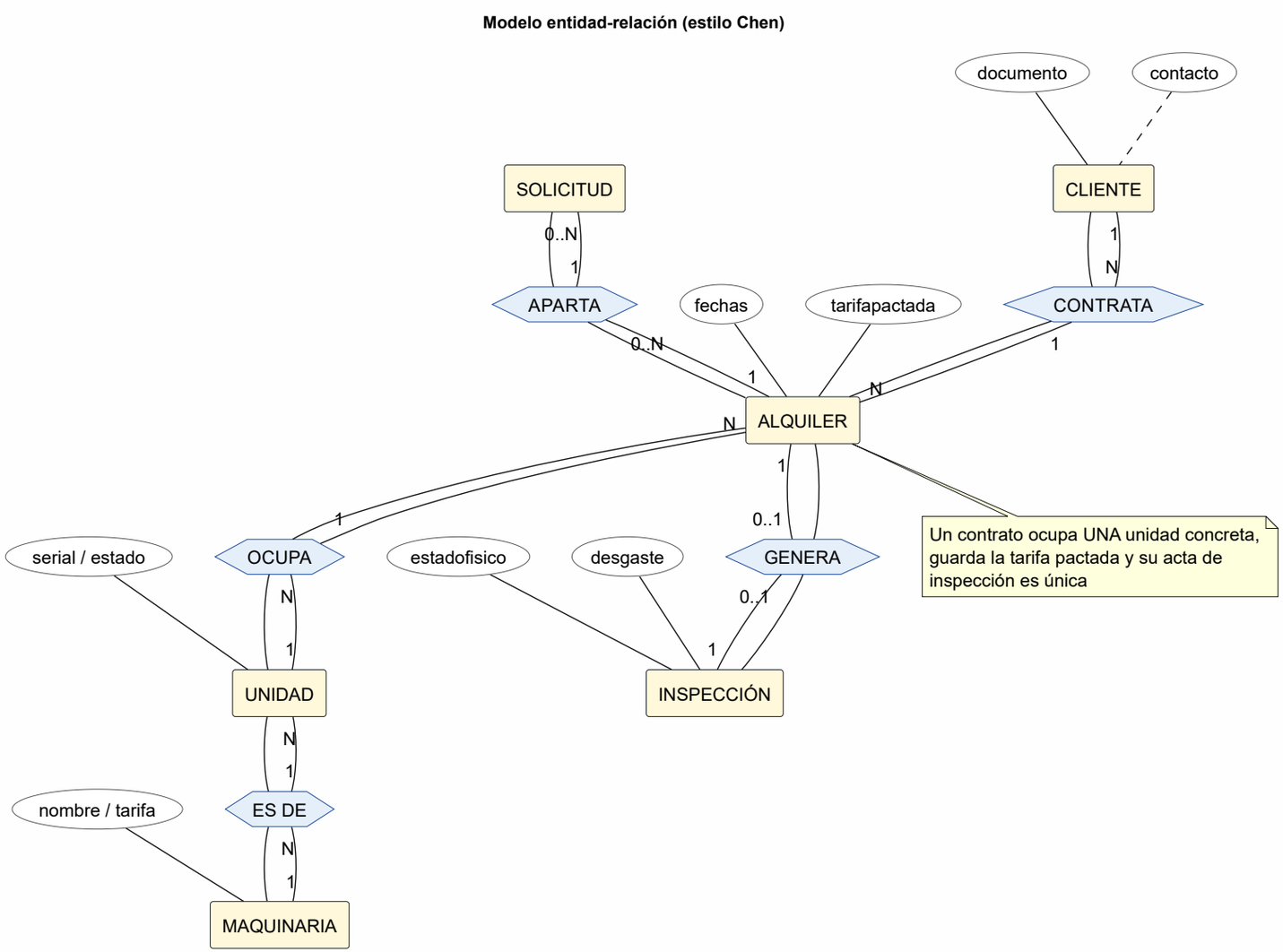


Figura 7

Modelo entidad-relación (estilo Chen): entidades, relaciones y atributos.

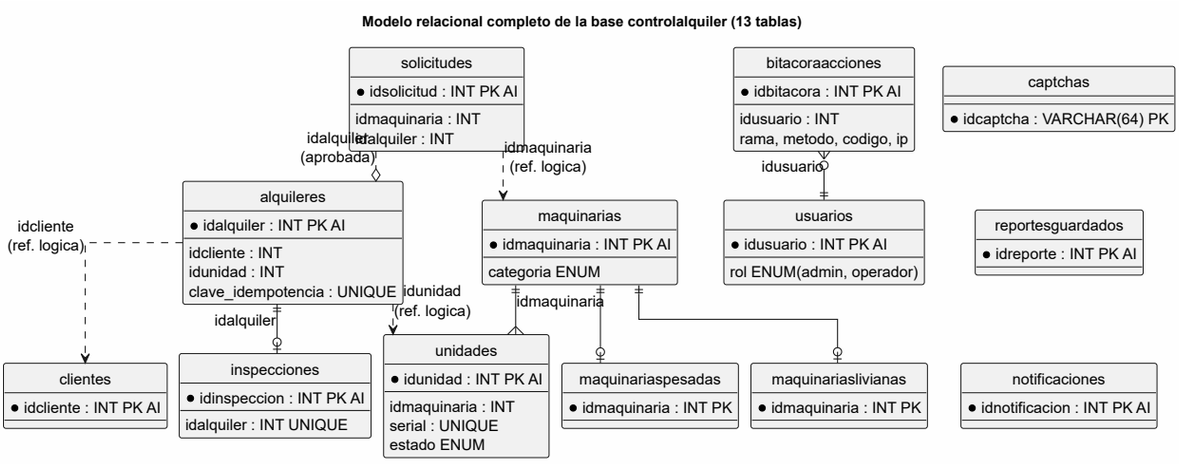


Figura 8

Modelo relacional completo: 13 tablas, llaves foráneas internas y referencias lógicas entre dominios.

4.2 Normalización hasta la tercera forma normal

La Tabla 7 aplica las tres formas normales al esquema y responde, una a una, las observaciones de la retroalimentación; concretamente, la 3FN elimina la dependencia transitiva que antes replicaba `idmaquinaria` en `inspecciones` y los contadores redondeados que se guardaban en dos lugares.

Tabla 7

Aplicación de las formas normales y respuesta a la retroalimentación.

Forma	Regla aplicada	Resultado en el esquema
1FN	Todos los atributos son atómicos	Ninguna tabla guarda listas ni JSON; las fotos son archivos en disco referenciados por carpeta.
2FN	Sin dependencias parciales de la clave	`maquinariaspesadas` y `maquinariaslivianas` separan los atributos que solo aplican a cada subtipo; sus claves son la PK de la madre.
3FN	Sin dependencias transitivas	`inspecciones` guardaba `idmaquinaria`, deducible de `idalquiler` → `idunidad` → `idmaquinaria`; el atributo se eliminó. El contador `disponibles` (duplicado en MySQL y JSON) también desapareció: se computa con `COUNT` al leer. Los históricos que sí persisten, porque son hechos del contrato y no derivados actuales, son `tarifa_pactada` y `valorreparacion` (CHECK ≥ 0). `reportesguardados` almacena una instantánea histórica intencional: es una desnormalización controlada, documentada y no una violación sin justificar.

Las referencias entre dominios (el par `idcliente` e `idunidad` en `alquileres`; el `idmaquinaria` en `solicitudes`) no se declaran como claves foráneas por diseño deliberado: la escritura de cada tabla corresponde a una sola API y la validez se garantiza por HTTP y transacción, no por integridad referencial entre dominios. La Tabla 8 detalla qué compensa físicamente esa decisión.

Tabla 8

Mecanismos compensatorios por no usar claves foráneas entre dominios.

Mecanismo implementado	Qué garantiza
Validación previa en la API escritora	El `id` apuntado existe al crear (por ejemplo, el cliente se busca antes de insertar el alquiler).
Transacción ACID con `BEGIN…COMMIT` y `ROLLBACK`	Creates y updates van a la fila, todos o nada: la unidad cambia e revisada al mismo instante que el contrato.
`SELECT…FOR UPDATE`	La misma unidad no puede reservarse dos veces: la segunda reserva espera el bloqueo y recibe 409.
Clave de idempotencia UNIQUE	El reenvío de un mismo contrato no duplica filas.

Mecanismo implementado	Qué garantiza
Borrado lógico (`activo=0`)	Ninguna maquinaria desaparece mientras tenga contratos activos: el historial conserva la referencia.
CHECK y ENUM en el script	Los estados y montos tienen dominio cerrado, sin texto libre.
Prueba E2E de integridad	Los escenarios límite (ocupada 409, retraso, duplicados) se ejecutan con cada entrega.

4.3 Diccionario de datos

La Tabla 9 resume el diccionario por tabla (campos relevantes, tipo, llave, CHECK/ENUM); el detalle fino de larguras y CONSTRAINTS está en el script de la sección siguiente. Esta tabla proviene directamente del SQL real, no de la documentación de diseño, por lo que no puede divergir del esquema ejecutado.

Tabla 9

Diccionario de datos (resumen por tabla).

Tabla	Campos y tipos	Llave	Reglas (CHECK/ENUM)
clientes	idcliente INT AI; nombre V100; documento V20; telefono V20; correo V100; direccion V150; fecharegistro DATE	idcliente PK	documento UNIQUE; CHECK teléfono o correo
usuarios	idusuario INT AI; usuario V60; nombre V100; clave V255 (bcrypt); rol ENUM; intentos, bloqueo, debe_cambiar_clave	idusuario PK	usuario UNIQUE; roles admin/operador/lector
bitacoraacciones	idbitacora INT AI; fecha DT; idusuario INT (referencia lógica); rama V30; metodo V10; codigo SMALLINT; ip V45	idbitacora PK	índices por fecha y usuario; purga a los 180 días
captchas	idcaptcha V64 PK; respuesta V30; expira DT; usado T1; ip V45	idcaptcha PK	un solo uso; purga de expiradas
alquileres	idalquiler INT AI; idcliente; idunidad; fechalquiler, devolución programada y real; tarifa_pactada, costo_movilizacion, valorreparacion DECIMAL(12,2); observaciones V250; clave UNIQUE V64	idalquiler PK; clave UNIQUE	CHECK fechas coherentes y montos ≥ 0
inspecciones	idinspeccion INT AI; idalquiler UNIQUE FK; estadofisico ENUM(3 niveles); niveldesgaste ENUM(4 niveles); descripcion-danos y observaciones V250	idinspeccion PK	una acta por contrato (ON DELETE RESTRICT)
solicitudes	idsolicitud INT AI; nombre, documento, contacto; idmaquinaria; fechas; estado ENUM(3 estados); motivo_rechazo V250; idalquiler NULL; fecha_creacion DT	idsolicitud PK	CHECK contacto y fechas; índices por estado y fecha
maquinarias	idmaquinaria INT AI; nombre V100 UNIQUE; categoria ENUM(pesada/liviana); carpeta V50 UNIQUE; tarifadiaria DECIMAL(12,2)>0; activo T1	idmaquinaria PK	índice por categoría+activo
maquinariaspesadas	idmaquinaria PK/FK; peso_toneladas DECIMAL(5,2)>0; costo_movilizacion DECIMAL(12,2)≥0	idmaquinaria PK	ON DELETE CASCADE (la baja es lógica en la madre)

Tabla	Campos y tipos	Llave	Reglas (CHECK/ENUM)
maquinariaslivianas	idmaquinaria PK/FK; ancho_metros DECIMAL(4,2)>0; apta_zona_urbana T1	idmaquinaria PK	CHECK de valores positivos
unidades	idunidad INT AI; idmaquinaria FK; serial V50 UNIQUE; estado ENUM(4 estados); estadodesde DT	idunidad PK	índice por (idmaquinaria, estado)
reportesguardados	idreporte INT AI; fecha DT; titulo V150; contadores INT; ingresos DECIMAL(12,2); máquina mas alquilada V100; contadores por familia	idreporte PK	CHECK no negativos
notificaciones	idnotificacion INT AI; titulo V150; mensaje V400; rama V30; nivel ENUM(info/exito/aviso/error); leida T1; fecha DT	idnotificacion PK	índices por leida y fecha

4.4 Script de creación de la base de datos

El script completo que se incluye aquí es el mismo que ejecuta `iniciar.ps1` en cada arranque (con sustitución de los marcadores `@CLAVE_*@` por claves fuertes generadas al vuelo y, en modo compartido, sin rotar el `root` de la instancia MariaDB), por ello el documento y el instanciado siempre coinciden. Presenta las 13 tablas con llaves, índices, ENUM y CHECK, sin comentarios ni datos semilla: solo la creación.

```
CREATE DATABASE IF NOT EXISTS controlalquiler CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE controlalquiler;

CREATE TABLE IF NOT EXISTS clientes (
  idcliente INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  documento VARCHAR(20) NOT NULL UNIQUE,
  telefono VARCHAR(20) NULL,
  correo VARCHAR(100) NULL,
  direccion VARCHAR(150) NULL,
  fecharegistro DATE NOT NULL,
  CHECK (telefono IS NOT NULL OR correo IS NOT NULL),
  INDEX ix_clientes_correo (correo)
);

CREATE TABLE IF NOT EXISTS usuarios (
  idusuario INT AUTO_INCREMENT PRIMARY KEY,
  usuario VARCHAR(60) NOT NULL UNIQUE,
  nombre VARCHAR(100) NOT NULL,
  clave VARCHAR(255) NOT NULL,
  rol ENUM('admin', 'operador') NOT NULL DEFAULT 'operador',
  activo TINYINT(1) NOT NULL DEFAULT 1,
  intentos_fallidos INT NOT NULL DEFAULT 0,
  bloqueado_hasta DATETIME NULL,
  debe_cambiar_clave TINYINT(1) NOT NULL DEFAULT 0,
  fecharegistro DATE NOT NULL,
  CHECK (activo IN (0, 1) AND intentos_fallidos >= 0 AND debe_cambiar_clave IN (0, 1))
);

CREATE TABLE IF NOT EXISTS bitacoraacciones (
  idbitacora INT AUTO_INCREMENT PRIMARY KEY,
  fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  idusuario INT NULL,
  rama VARCHAR(30) NOT NULL,
  metodo VARCHAR(10) NOT NULL,
  codigo SMALLINT NOT NULL,
  ip VARCHAR(45) NOT NULL,
  INDEX ix_bitacora_fecha (fecha),
  INDEX ix_bitacora_usuario (idusuario)
```

```
);

CREATE TABLE IF NOT EXISTS captchas (
  idcaptcha VARCHAR(64) PRIMARY KEY,
  respuesta VARCHAR(30) NOT NULL,
  expira DATETIME NOT NULL,
  usado TINYINT(1) NOT NULL DEFAULT 0,
  ip VARCHAR(45) NOT NULL,
  creado DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX ix_captchas_expira (expira)
);

CREATE TABLE IF NOT EXISTS alquileres (
  idalquiler INT AUTO_INCREMENT PRIMARY KEY,
  idcliente INT NOT NULL,
  idunidad INT NOT NULL,
  fechalquiler DATE NOT NULL,
  fechadevolucionprogramada DATE NOT NULL,
  fechadevolucionreal DATE NULL,
  tarifa_pactada DECIMAL(12,2) NOT NULL,
  costo_movilizacion_pactado DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  valorreparacion DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  observaciones VARCHAR(250) NULL,
  clave_idempotencia VARCHAR(64) NOT NULL UNIQUE,
  CHECK (fechadevolucionprogramada >= fechalquiler),
  CHECK (tarifa_pactada > 0 AND costo_movilizacion_pactado >= 0 AND valorreparacion >= 0),
  INDEX ix_alquileres_unidad (idunidad, fechadevolucionreal)
);

CREATE TABLE IF NOT EXISTS inspecciones (
  idinspeccion INT AUTO_INCREMENT PRIMARY KEY,
  idalquiler INT NOT NULL UNIQUE,
  estadofisico ENUM('bueno', 'regular', 'malo') NOT NULL,
  niveldesgaste ENUM('ninguno', 'bajo', 'medio', 'alto') NOT NULL,
  descripciondanos VARCHAR(250) NULL,
  observaciones VARCHAR(250) NULL,
  FOREIGN KEY (idalquiler) REFERENCES alquileres(idalquiler) ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS solicitudes (
  idsolicitud INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  documento VARCHAR(20) NOT NULL,
  telefono VARCHAR(20) NULL,
  correo VARCHAR(100) NULL,
  idmaquinaria INT NOT NULL,
```

```
  fechalquiler DATE NOT NULL,
  fechadevolucionprogramada DATE NOT NULL,
  observaciones VARCHAR(250) NULL,
  estado ENUM('pendiente', 'aprobada', 'rechazada') NOT NULL DEFAULT 'pendiente',
  motivo_rechazo VARCHAR(250) NULL,
  idalquiler INT NULL,
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (telefono IS NOT NULL OR correo IS NOT NULL),
  CHECK (fechadevolucionprogramada >= fechalquiler),
  INDEX ix_solicitudes_estado (estado),
  INDEX ix_solicitudes_fecha (fecha_creacion)
);

CREATE TABLE IF NOT EXISTS maquinarias (
  idmaquinaria INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE,
  categoria ENUM('pesada', 'liviana') NOT NULL,
  carpeta VARCHAR(50) NOT NULL UNIQUE,
  tarifadiaria DECIMAL(12,2) NOT NULL,
  activo TINYINT(1) NOT NULL DEFAULT 1,
  CHECK (tarifadiaria > 0),
  INDEX ix_maquinarias_cat (categoria, activo)
);

CREATE TABLE IF NOT EXISTS maquinariaspesadas (
  idmaquinaria INT PRIMARY KEY,
  peso_toneladas DECIMAL(5,2) NOT NULL,
  costo_movilizacion DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  CHECK (peso_toneladas > 0 AND costo_movilizacion >= 0),
  FOREIGN KEY (idmaquinaria) REFERENCES maquinarias(idmaquinaria) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS maquinariaslivianas (
  idmaquinaria INT PRIMARY KEY,
  ancho_metros DECIMAL(4,2) NOT NULL,
  apta_zona_urbana TINYINT(1) NOT NULL DEFAULT 1,
  CHECK (ancho_metros > 0 AND apta_zona_urbana IN (0, 1)),
  FOREIGN KEY (idmaquinaria) REFERENCES maquinarias(idmaquinaria) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS unidades (
  idunidad INT AUTO_INCREMENT PRIMARY KEY,
  idmaquinaria INT NOT NULL,
  serial VARCHAR(50) NOT NULL UNIQUE,
  estado ENUM('disponible', 'alquilada', 'en_inspeccion', 'en_reparacion') NOT NULL DEFAULT 'disponible',
  estadodesde DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (idmaquinaria) REFERENCES maquinarias(idmaquinaria) ON DELETE RESTRICT,
  INDEX ix_unidades_maq (idmaquinaria, estado)
);

CREATE TABLE IF NOT EXISTS reportesguardados (
  idreporte INT AUTO_INCREMENT PRIMARY KEY,
  fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  titulo VARCHAR(150) NOT NULL,
  total_contratos INT NOT NULL DEFAULT 0,
  contratos_activos INT,
  contratos_morosos INT,
  ingresos DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  maquina_mas_alquilada VARCHAR(100) NULL,
  unidades_pesadas INT NOT NULL DEFAULT 0,
  disponibles_pesadas INT NOT NULL DEFAULT 0,
  unidades_livianas INT NOT NULL DEFAULT 0,
  disponibles_livianas INT NOT NULL DEFAULT 0,
  CHECK (total_contratos >= 0 AND ingresos >= 0),
  INDEX ix_reportes_fecha (fecha)
);

CREATE TABLE IF NOT EXISTS notificaciones (
  idnotificacion INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(150) NOT NULL,
  mensaje VARCHAR(400) NOT NULL DEFAULT '',
  rama VARCHAR(30) NOT NULL DEFAULT '',
  nivel ENUM('info', 'exito', 'aviso', 'error') NOT NULL DEFAULT 'info',
  leida TINYINT(1) NOT NULL DEFAULT 0,
  fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (CHAR_LENGTH(titulo) BETWEEN 1 AND 150),
  CHECK (CHAR_LENGTH(mensaje) <= 400),
  INDEX ix_notif_leida (leida),
  INDEX ix_notif_fecha (fecha)
);
```

5 Especificación técnica de las piezas

Cada pieza del sistema se especifica aquí con su responsabilidad exclusiva de escritura, su tecnología, sus ficheros reales y su contrato de endpoints: el cliente web (SPA), el gateway con sus doce filtros, la API PHP de identidad y agenda, la API Node de contratos, la API Python de catálogo y reportes, y la tabla de dueños de tablas que gobierna toda lectura y escritura. El capítulo también cierra con el contrato completo de ramas del gateway, para que un evaluador pueda verificar, método por método, qué hace cada pieza sin leer el código fuente, y con las librerías con versiones exactas entregadas. Tres reglas transversales protegen esta frontera: cada API valida la cabecera interna `X-Clave-Interna` comparada en tiempo constante, exige que la conexión llegue por `127.0.0.1` y verifica la cabecera `Host`; nadie fuera del loopback habla con un servicio de dominio, y la base de datos recibe solo usuarios con privilegio mínimo por pieza. cada sección de API sigue el mismo esquema de lectura: una explicación en palabras sencillas de para qué sirve la pieza (qué encarga y qué no toca), la lista completa de sus endpoints reales con método, ruta interna y efecto, y el detalle de construcción (framework, pool de base de datos, validaciones y cabeceras internas). Así se evita la documentación que presupone experiencia previa: quien no conoce Express, PHP ni FastAPI puede entender qué hace cada pieza leyendo el párrafo inicial de cada tabla, y quien sí conoce las herramientas tiene el contrato exacto de cada ruta para probarla contra el sistema local.

5.1 Cliente web (SPA en localhost:8443)

La interfaz es **JavaScript vanilla ES6+ con Bootstrap 5**, servida por el archivo `server.js` de la carpeta `frontend-server` (HTTPS, CSP, HSTS, nosniff, SRI, gzip con ETag/304 y anti-*path traversal* de Windows). Solo habla con el gateway (`https://localhost:3000/api`) en un único formato `{TOKEN, dat:{RAMA, METODO, DATOS}}` y el token vive solo en memoria (`TOKEN_MEMORIA`), nunca en `localStorage`. Sus módulos y responsabilidades son: `config/api/funciones` (cliente HTTP y utilidades), `inicio/login` (vista pública-privada, captcha y roles), `dashboard` (4 gráficas vivas), `calendario` (FullCalendar) y `notificaciones` (campana con polling de 60 s). Los módulos de negocio —maquinaria, clientes, unidades, alquileres, devoluciones, reportes, solicitudes, clave y usuarios— comparten estilos propios para claro/oscuro e impresión.

5.2 API Gateway (Node/Express, 3000)

`api-gateway` con `express.json(1MB)`, CORS de orígenes exactos, TLS 1.2 mínimo, verificación de Host (anti-*DNS rebinding*) y **doce filtros**: (1) loopback; (2) cabeceras de seguridad; (3) Host permitido; (4) 100 req/min/IP; (5) estructura `{TOKEN,dat}` con rechazo de claves peligrosas; (6) bloqueo-login 5/15 min; (7) fotos 20/15 min; (8) captcha 15/5 min; (9) solicitudes públicas 5/15 min; (10) JWT HS256 (iss/aud/exp) con ramas públicas declaradas; (11) matriz de permisos (admin *, operador 21 acciones; el rol de solo lectura fue retirado del código, ver sección 3); (12) `procesar`: *login* firma el JWT y el resto `resolverDestino(rama,metodo,datos)` con mapa cerrado (anti-SSRF) y reenvío con cabecera secreta `X-Clave-Interna`. Las escrituras de negocio dejan bitácora (fire-and-forget a PHP).

La Figura 9 documenta el gateway respondiendo en vivo frente al navegador: es una petición real del SPA (la del catálogo público) vista mientras se ejecuta, con su envoltorio `{ok, codigo, mensaje, datos}` y las doce maquinarias sembradas entregando tarifa, categoría, peso, movilización y contadores de unidades calculados al vuelo.

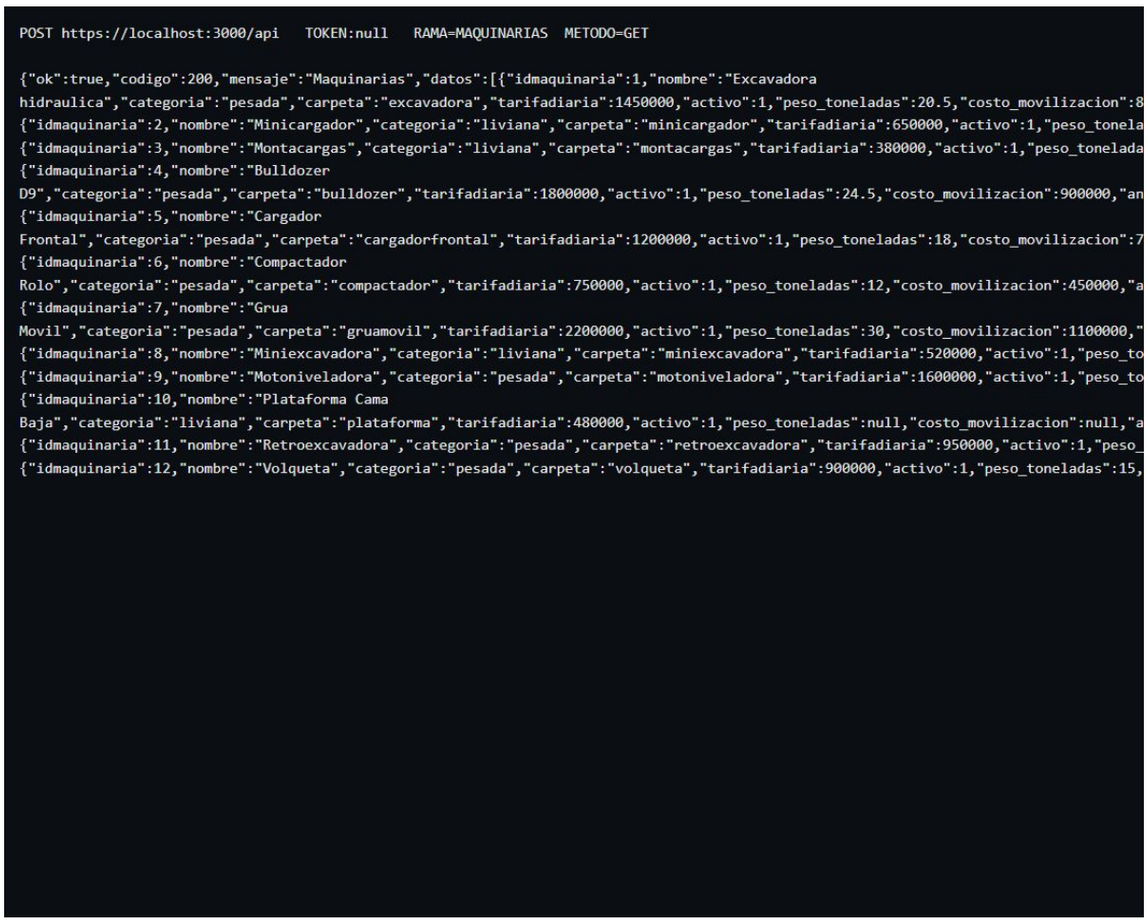


Figura 9

Respuesta real del gateway en el navegador: rama MAQUINARIAS con código 200.

5.3 API PHP (8000)

En palabras sencillas: la API PHP es la *puerta de seguridad* del sistema. Imaginarla como el responsable que revisa a quién se le permite entrar: primero pediste resolver un pequeño acertijo visual (el captcha: “haz clic en la imagen correcta”), y después comprueba usuario y contraseña —cada contraseña guardada como un mugrom digest bcrypt que ni siquiera el sistema puede invertir—, controla cuántos intentos fallidos lleva cada cuenta y entrega el “pase” (el token) con el que se recorre el resto del panel. También lleva la carpeta de proveedores (clientes), la agenda de cuentas nuevas y la bitácora donde se anota quién hizo qué. *Nada de líneas de negocio*: no sabe lo que es un alquiler; solo reconoce a la gente.

Cómo está construida: un front-controller único (`enrutador.php` sobre el servidor nativo de PHP 8 de XAMPP, puerto 8000) que carga su `.env`, exige loopback, valida el Host y comprueba la cabecera interna `X-Clave-Interna` comparándola en tiempo constante. Solo esa API escribe en las tablas de identidad: `usuarios`, `captchas`, `bitacoraacciones` y `clientes`. Sus consultas son 100 % preparadas (MySQLi) y sus cuerpos de middleware `htmlspecialchars` para XSS.

Tabla 10

Endpoints reales de la API PHP.

Método	Ruta interna	Qué hace
POST	/captcha/generar	crea el captcha de un solo uso con cuota de 15 por 5 min por IP
POST	/usuarios/login	valida captcha, bcrypt y bloqueo; devuelve los datos de sesión
PUT	/usuarios/clave	cambia la clave propia (exige la actual, mínimo 12 caracteres)
PATCH	/usuarios/clave-admin	reset de la clave de otro usuario (solo administrador)
GET	/usuarios/lista	lista de cuentas (rama CUENTAS, solo administrador)
POST	/usuarios/crear	crea una cuenta de operador con clave temporal obligatoria a cambiar
GET	/clientes[/id]	busca clientes con LIKE y comodines escapados
POST	/clientes	registra un cliente con documento único y contacto válido
POST	/bitacora	guarda el registro de auditoría generado por el gateway

5.4 API Node (3002)

En palabras sencillas: la API Node es el *buchón de las obras*: maneja el dinero y las fechas. Cuando alguien quiere una máquina, esta API revisa con lupa que la unidad concreta esté libre (le "apunta el dedo" con `FOR UPDATE` para que nadie más la toque en ese instante), escribe el contrato, y si dos personas intentan lo mismo al mismo tiempo, una gana y la otra recibe un claro "ya está ocupada" (409). Cuando la máquin se devuelve, levanta el acta de inspección calcula cuántos días extra hubo —a la tarifa pactada del contrato, no a la de hoy— y avisa a todo el equipo por la campana. Es la única pieza que puede tocar las tablas de `alquileres`, `inspecciones`, `solicitudes` y `notificaciones`.

Cómo está construida: `servidor.js` con Express y pool `mysql2/promise` (`dateStrings:true` para las fechas sin desfase horario y `max_statement_time` como apagado de emergencia de consultas lentas). Toda operación de escritura corre dentro de `BEGIN…COMMIT` con `ROLLBACK`; la idempotencia protege contra doble envío; sin CORS: solo el gateway habla con ella.

Tabla 11

Endpoints reales de la API Node.

Método	Ruta interna	Qué hace
GET	/alquileres[/id]	historial con los cobros calculados al leer
POST	/alquileres	crea el contrato en transacción con `FOR UPDATE` e idempotencia (201)
PUT	/alquileres/{id}/devolucion	registra entrega, inspección única, calcula retraso/totales y estado final
GET	/solicitudes[/id]	lista o detalle de solicitudes con filtros
POST	/solicitudes	recibe la solicitud del sitio público (201)
PUT	/solicitudes/{id}/aprobar	crea el contrato con `FOR UPDATE` dentro de la misma transacción
PUT	/solicitudes/{id}/rechazar	rechaza con motivo obligatorio
GET	/notificaciones/lista	lista los últimos avisos y el contador de no leídos
PUT	/notificaciones/{id}/leida	marca un aviso como leído
PUT	/notificaciones/leer-todas	marca todas (`todas:true`)

5.5 API Python (3003)

En palabras sencillas: la API Python es la *vitrina y los mapas de la empresa*. Mantiene el catálogo que todos ven (qué máquinas hay, su foto, su tarifa por día, si hay unidades libres) y además guarda las *estadísticas*: cuándo black un negocio va bien ===cuántos contratos, cuántos morosos, cuánto se ingresó— y lo deja en un reporte que se puede guardar y después imprimir. También es la guardiana de las llaves de las familias de máquinas (pesadas y livianas) y de las unidades concretas con su serial.

Cómo está construida: `servidor.py` con FastAPI sin `/docs` y pool de `mysql-connector`; validación con Pydantic y `magic bytes` reales para aceptar solo JPG/PNG verdaderos de hasta 2 MB, nombre de archivo aleatorio y carpeta fuera del webroot (anti-*path traversal* con `commonpath`). Las rutas de administrador exigen la cabecera interna `x-rol` como frontera de confianza adicional.

Tabla 12

Endpoints reales de la API Python.

Método	Ruta interna	Qué hace
GET	/maquinarias	catálogo con filtros y disponibilidad calculada al vuelo (público)
POST	/maquinarias	crea máquina con foto validada y unidades en lote (201)
PATCH	/maquinarias/{id}	edita tarifa, movilización, agrega unidades o desactiva (bloqueado si hay ocupadas)
DELETE	/maquinarias/{id}	baja lógica (solo administrador)
POST	/unidades/pormaquina	agrega unidades con serial único a una máquina
PUT	/unidades/{id}/estado	cambia estado con reglas (400 ante saltos prohibidos)
POST	/fotos	sube una imagen real validada (200, o 400 ante trampa)
POST	/reportes/resumen	construye el resumen del negocio con los totales vivos
GET, POST	/reportes	lista y guarda reportes del equipo
DELETE	/reportes/{id}	borra un reporte guardado (solo administrador)

5.6 Dueño de las tablas y librerías

La Tabla 13 recapitula la depósito de escritura por tabla y el módulo del panel que la consume; Tabla 14 lista librerías y versiones reales.

Tabla 13

Escritura por tabla y consumo por módulo.

Tabla(s) escritas	Dueño	Uso en el panel
clientes, usuarios, captchas, bitácora	PHP	login, claves/cuentas, clientes, catálogo (válida el cliente del contrato)
alquileres, inspecciones, solicitudes, notificaciones	Node	alquileres, devoluciones, solicitudes, calendario, dashboard y campana
maquinarias (+pesadas/livianas), unidades	Python	catálogo, administrar, unidades, calendario
reportesguardados	Python	reportes y dashboard

Tabla 14

Librerías y dependencias reales.

Pieza	Librerías
Frontend	Bootstrap 5.3.3 (SRI), Bootstrap Icons 1.13.2, Chart.js 4.4.7, FullCalendar 6.1.15 + locale es, fuentes propias sin CDN
Gateway	express 4.19, cors 2.8, dotenv 16.4, jsonwebtoken 9.0
API Node	express 4.19, mysql2 3.9.7, dotenv 16.4
API PHP	mysqli (XAMPP) + password_hash nativo
API Python	fastapi 0.115.6, uvicorn 0.34, mysql-connector-python 9.1, pydantic 2.10
Base de datos	MariaDB (XAMPP), motor InnoDB, `utf8mb4_unicode_ci`

5.7 Contrato del API Gateway: ramas, dueños y métodos

Toda petición del cliente entra por `POST /api` con la estructura `{TOKEN, dat:{RAMA, METODO, DATOS}}` y todas las respuestas comparten el envoltorio `{ok, codigo, mensaje, datos}`. La Tabla 15 es el contrato cerrado entre las cuatro piezas (es literalmente el objeto `RAMAS` del archivo `validacion.js`): cada rama pertenece a una sola API dueña con su lista de métodos permitidos. Cualquier rama o método fuera de esa lista responde 404 o 405, y como el destino se resuelve con un mapa estático (no hay rutas construidas con datos del cliente), el gateway no puede ser usado para SSRF.

Tabla 15

Ramas del gateway por dueño y métodos permitidos.

Rama	Dueño	Métodos permitidos
USUARIOS	PHP	POST (login), PUT (clave propia), PATCH (reset admin)
CUENTAS	PHP	GET (lista), POST (crear)
CLIENTES	PHP	GET, POST
CAPTCHA	PHP	POST (generar)
ALQUILERES	Node	GET, POST
DEVOLUCION	Node	PUT, GET
SOLICITUDES	Node	GET, POST, PUT (aprobar/rechazar)
NOTIFICACIONES	Node	GET, PUT (marcar leídas)
MAQUINARIAS	Python	GET (público), POST, PATCH, DELETE (baja lógica)
UNIDADES	Python	POST (por máquina), PUT (estado)
FOTOS	Python	POST
REPORTES	Python	GET, POST (resumen o guardar), DELETE

Una precisión importante sobre la bitácora: el gateway no es dueño de tablas; lo que hace es *generar* el evento de auditoría y persistirlo con un POST directo a `/bitacora` de la API PHP (firmado con la clave interna y de tipo *fire-and-forget*, es decir, no bloqueante). Si PHP estuviera caído, el evento queda en los registros del gateway pero no se persiste en la base; esa limitación queda declarada aquí de forma explícita.

6 Flujo del sistema paso a paso

Este capítulo sigue una petición de punta a punta: desde el formato del mensaje que obliga el gateway, el captcha obligatorio del acceso, el alquiler con bloqueo de fila e idempotencia, la devolución con acta y cobros calculados con la tarifa pactada, hasta las notificaciones que el equipo consume en vivo. Todos los saltos se ilustran con diagramas de secuencia regenerados y con estructuras JSON reales capturadas de la sesión de entrega, explicadas campo por campo para que la evidencia sea legible y trazable. Antes de cada paso conviene recordar el orden de validación que sufre la petición: verificación de `Host` y origen, cuota por IP, estructura del envoltorio, token y rol, y solo entonces el reenvío a la API dueña con la clave interna; si una capa no pasa, la respuesta regresa con el código correspondiente (400, 401, 403, 405, 409 o 429) sin llegar a tocar la base de datos. el capítulo incluye además la sección de reglas exactas de cálculo del cobro con un ejemplo numérico completo, porque en la versión anterior un lector podía confundirse sobre cuántas veces se cobraba el día de retraso: ahora la fórmula está escrita línea por línea (días de servicio inclusivos, días de retraso aparte, tarifa pactada como multiplicador y, al final, la suma de movilización y reparación); cada campo del JSON de una devolución se rastrea hasta su fórmula, y la corrección del doble cobro queda documentada sin esconder que existió.

6.1 Qué es la página y cómo se usa

El sistema tiene dos caras. La **pública** muestra el héroe con la propuesta de valor, cifras vivas, beneficios, catálogo filtrable con fotos, un formulario de solicitud con costo estimado en vivo y los datos de contacto; también abre el inicio de sesión con captcha. La **privada** es el panel de gestión: doce módulos alimentados exclusivamente por el gateway (ver Tabla 6), con notificaciones de la esquina, calendario mensual y capacidad de impresión. La Figura 10 muestra la vista pública y la Figura 11 su catálogo.

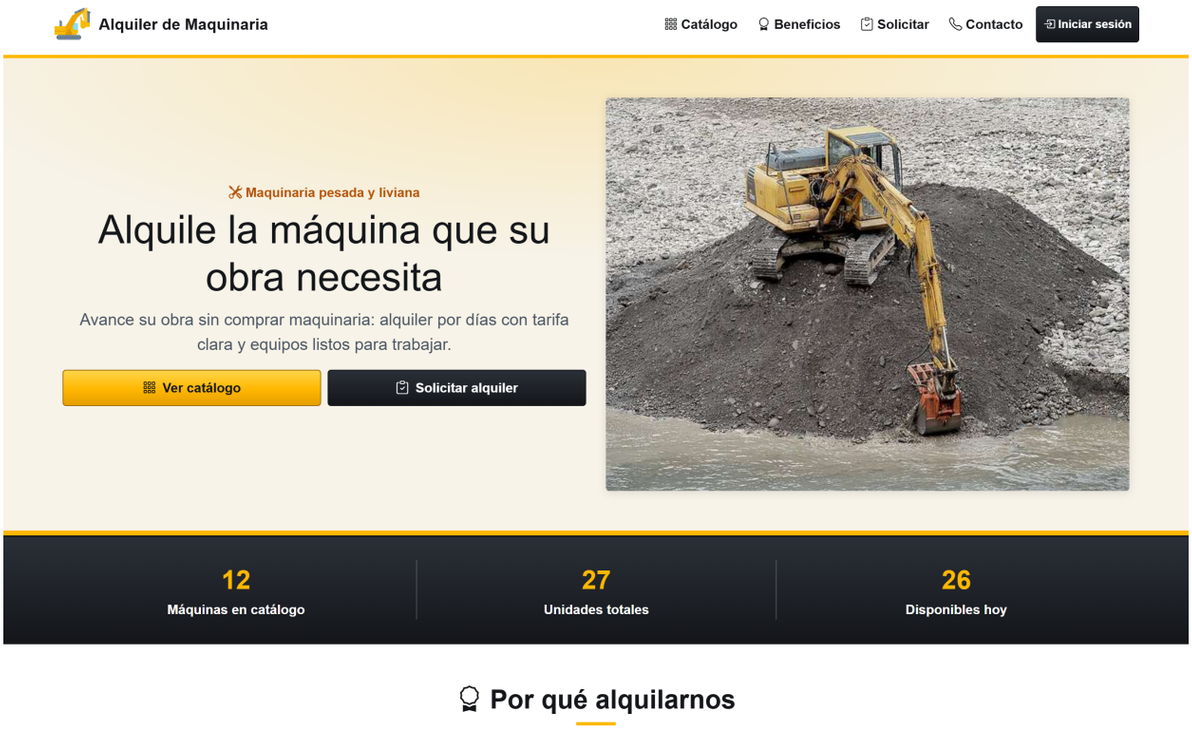


Figura 10

Sitio público recién cargado (vista de escritorio).

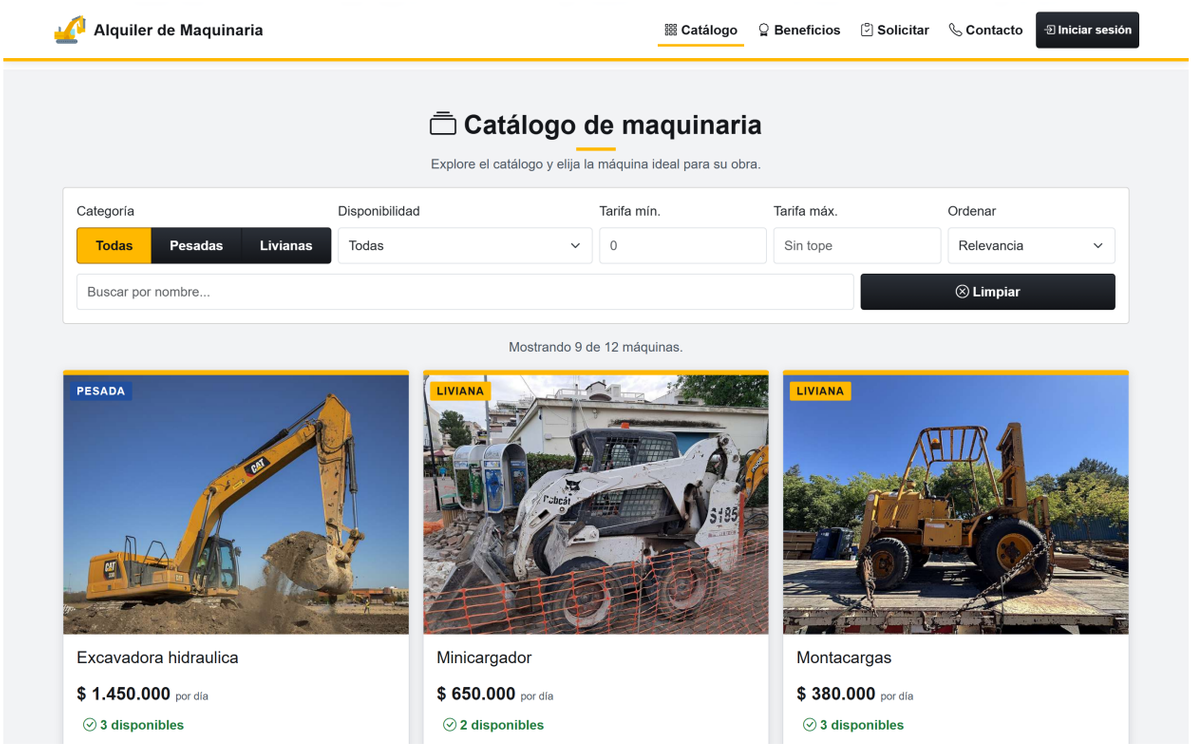


Figura 11

Catálogo público con filtros, orden y paginación.

6.2 El captcha: qué es y cómo funciona

El captcha de este sistema es **visual**: PHP elige aleatoriamente un objetivo (excavadora, grúa, etc.), guarda en la tabla `captchas` un identificador aleatorio de 32 caracteres, la respuesta, la IP, la fecha de expiración (+5 minutos) y un sello de uso (false); y devuelve al navegador cuatro imágenes (el objetivo y tres distractores que no comparten silueta con él; entre ellos la excavadora, el bulldozer o la grúa). El usuario debe hacer clic en el ícono correcto; en el envío de credenciales PHP verifica (existencia, no usado, no expirado, coincidencia en tiempo constante), marca el captcha como usado *suceda lo que suceda* y solo después evalúa credenciales. Si falla, responde 400 y el nonce muere. La Figura 12 muestra la secuencia completa del login con captcha y la emisión del token.

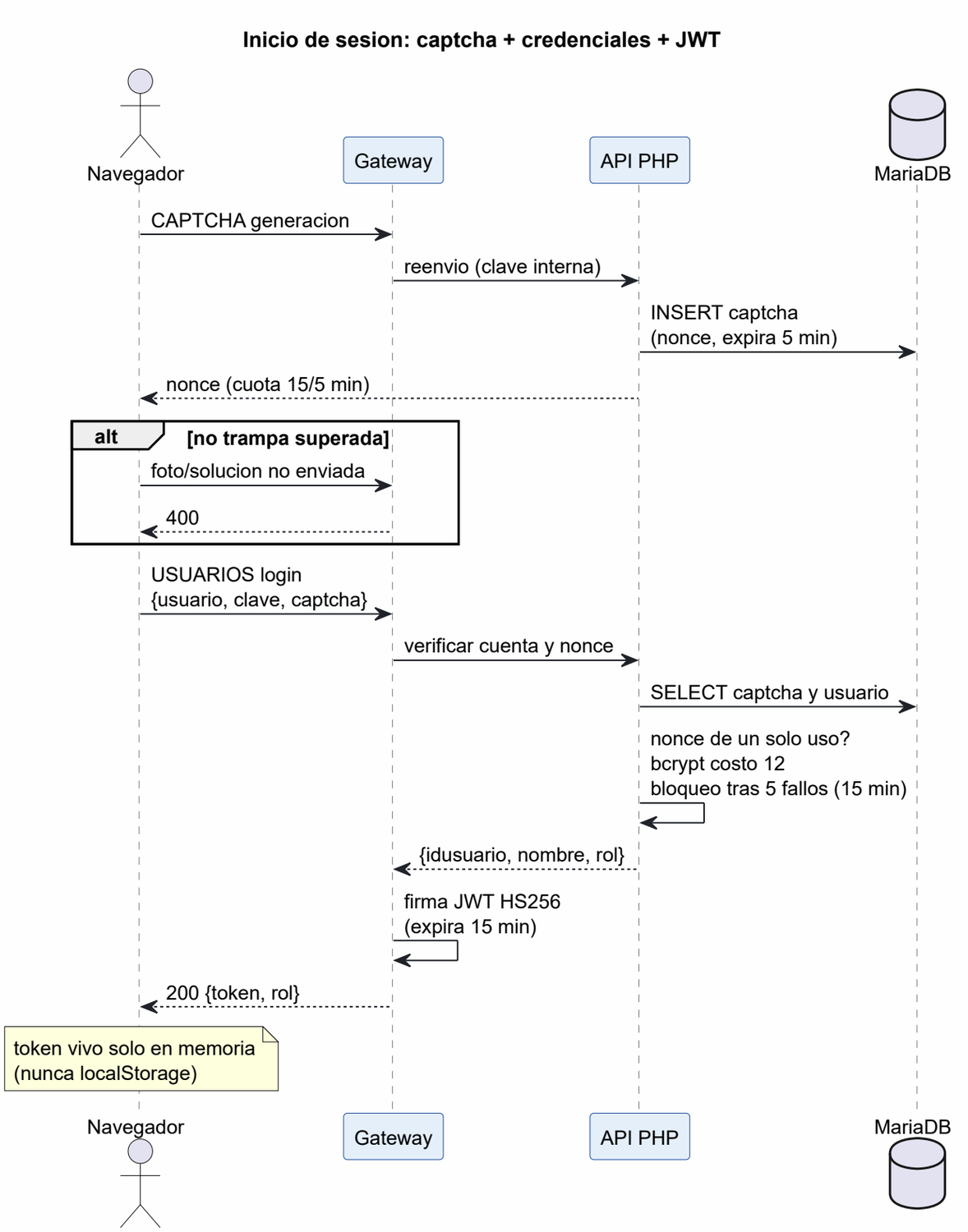


Figura 12

Secuencia de inicio de sesión con captcha y JWT.

6.3 El patrón API Gateway y su flujo

El gateway es un **front-controller**: el navegador no conoce las APIs de dominio, solo `POST /api` con estructura `{TOKEN, dat:{RAMA, METODO, DATOS}}`. Cada petición recorre los doce filtros enumerados en 5 y termina en un mapa `RAMA:METODO -> ruta interna` cerrado (no se puede ruteo dinámico ni *SSRF*); el reenvío lleva la cabecera `X-Clave-Interna` correspondiente, comparada en tiempo constante por la API objetivo, que a su vez reexige loopback y Host. Si la API muere, el gateway responde 502/504 sin colgar al navegador. Cada escritura queda registrada en la bitácora (fire-and-forget).

6.4 Flujo de crear alquiler y de devolución

La Figura 13 y la Figura 14 detallan los dos flujos críticos con bloqueo de fila, idempotencia y avisos. La creación exige unidad libre (`FOR UPDATE`), calcula días y totales e inserta la notificación; la devolución inserta la inspección (una por contrato), marca la fecha real, calcula días/retra- so/totales *al leer* con la **tarifa pactada** y sugiere el estado final de la unidad (disponible / en inspección / en reparación).

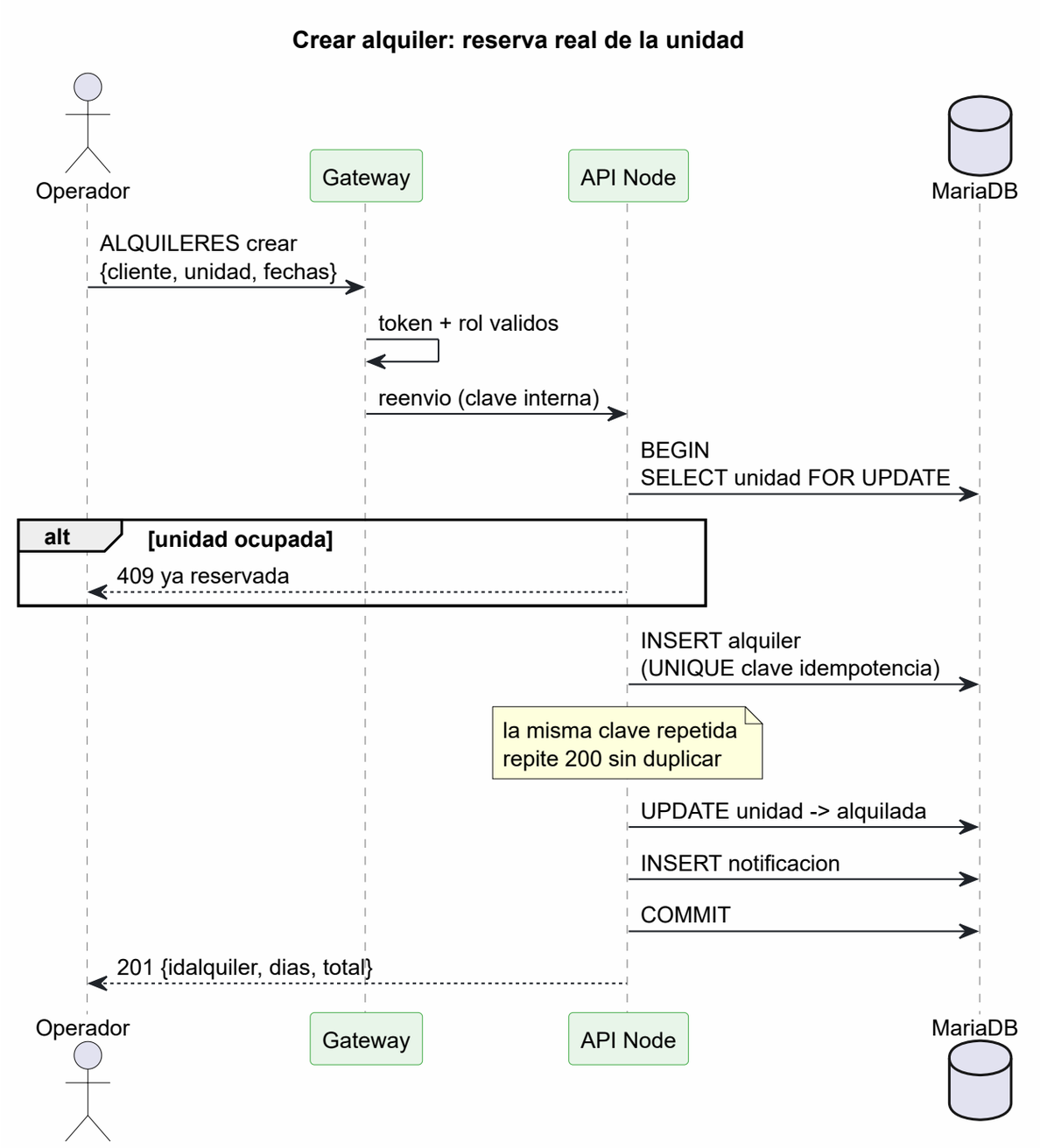


Figura 13

Secuencia de creación de contrato con transacción e idempotencia.

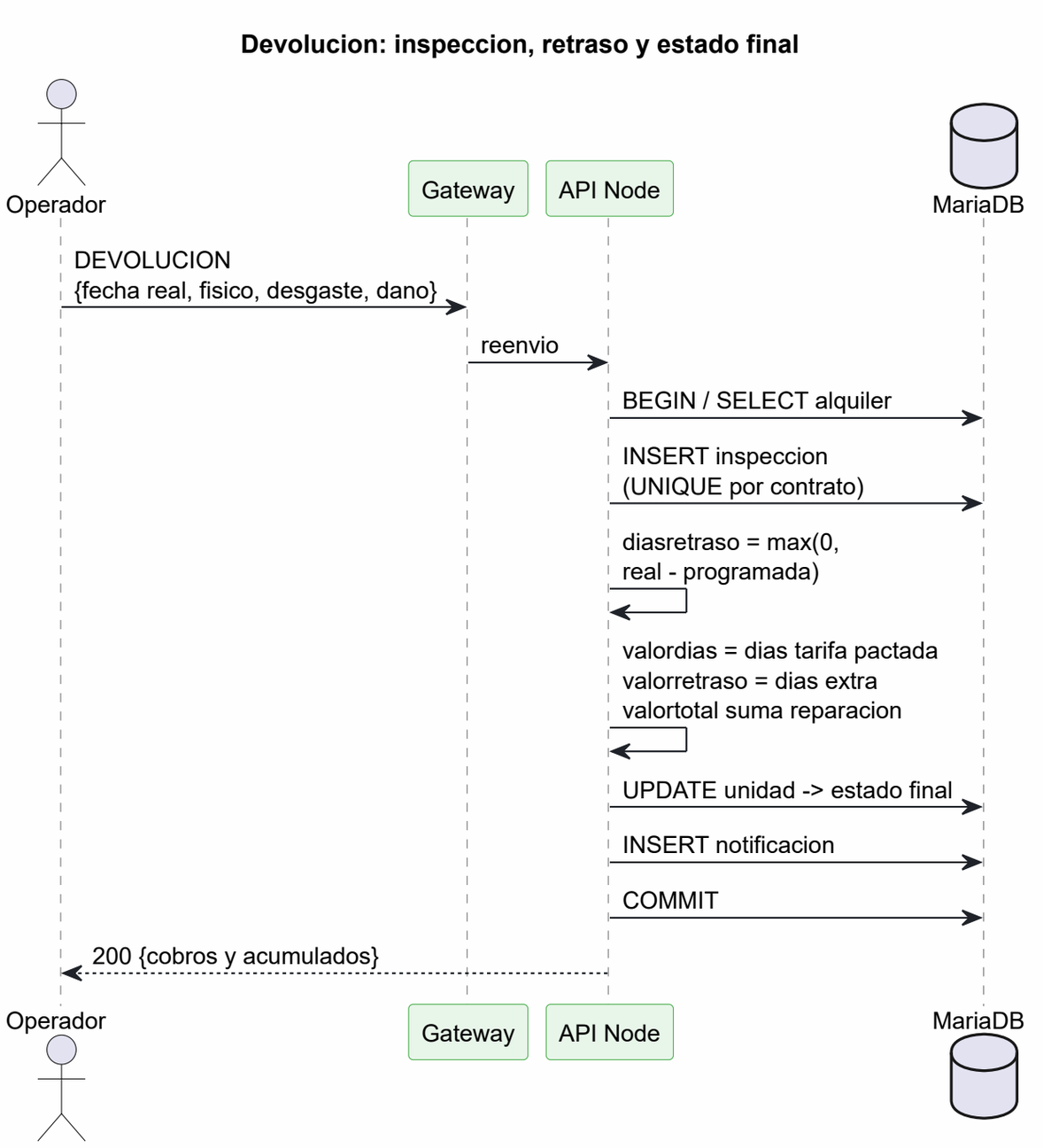


Figura 14

Secuencia de devolución con inspección y cobro final.

6.5 Estados de la unidad y notificaciones

La Figura 15 esquematiza la máquina de estados de una unidad (tal como la aplica el esquema y la API Python), y la Figura 16 el ciclo de vida de una notificación: generación automática por eventos de negocio, lectura con badge y *toasts*, y purga física automática a los 60 días (un `DELETE` liviano que la API Node ejecuta junto al arranque).

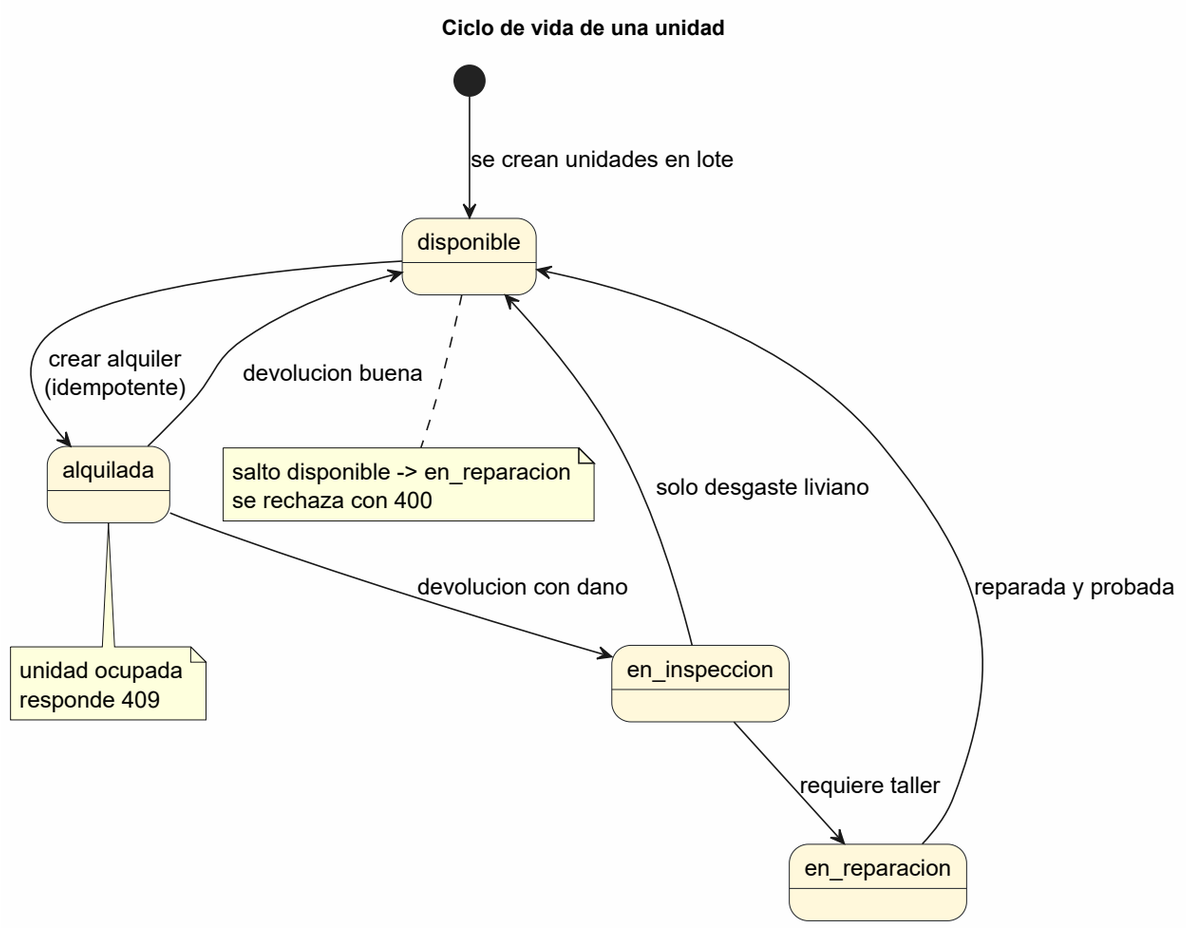


Figura 15

Máquina de estados de la unidad.

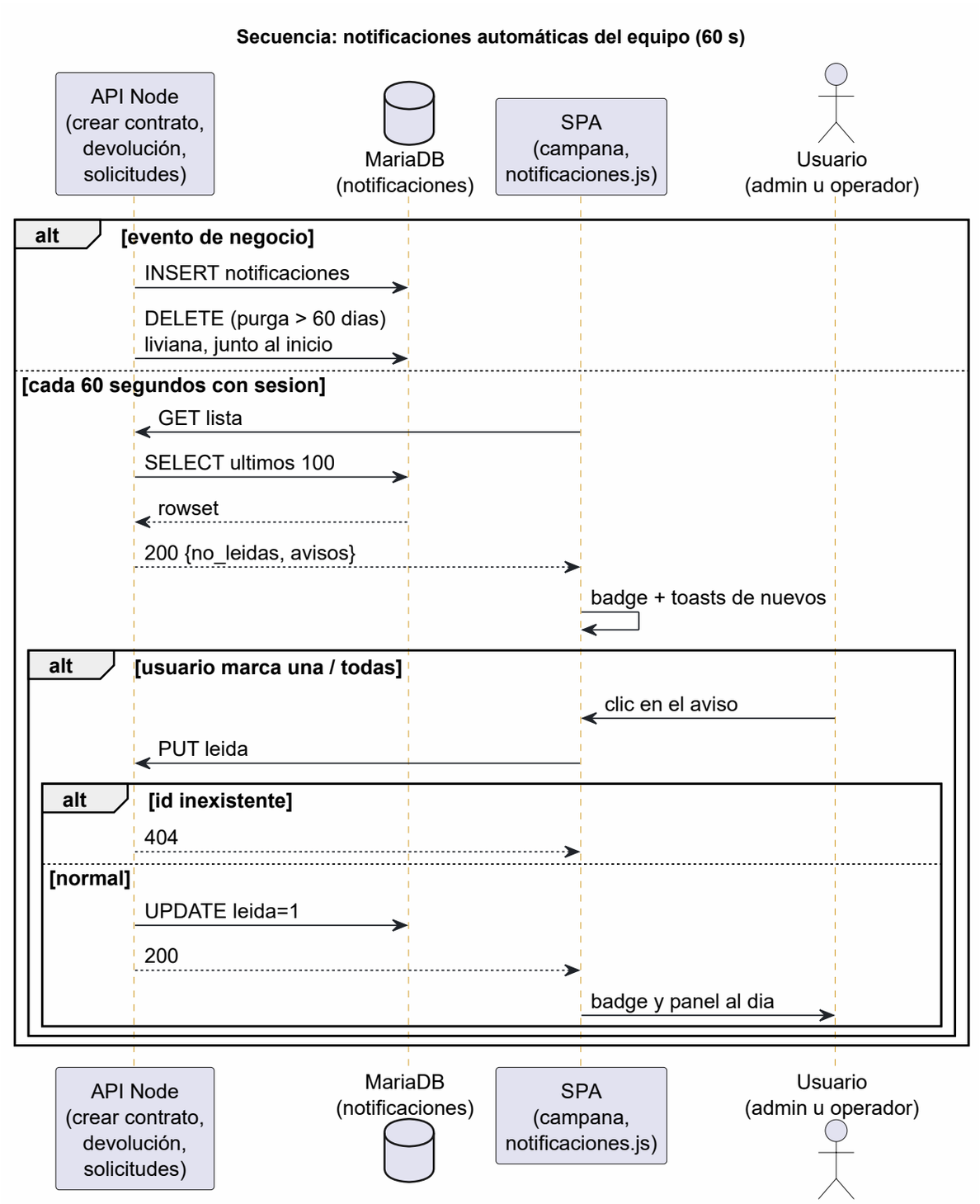


Figura 16

Ciclo de generación y consumo de notificaciones.

6.6 Evidencia viva: JSON del gateway explicado

Durante la sesión real se capturaron las respuestas del gateway en el navegador (capturas en vivo incluidas en esta sección); aquí se explican sus estructuras. La estructura común es `{ok, codigo, mensaje, datos}`: `ok` booleano, `codigo` HTTP duplicado en el cuerpo (útil para el SPA), `mensaje` acordado para los toasts y `datos` con la carga útil.

El catálogo público entrega por máquina `idmaquinaria, nombre, categoria, carpeta, tarifadiaria, activo, total_unidades y unidades_disponibles` (calculados al vuelo, sin contadores); con sesión el login entrega `idusuario, usuario, nombre, rol, debe_cambiar_clave, token` y el SPA guarda token y rol en memoria aplicando `data-rol` para mostrar u ocultar módulos. El historial expone los contratos con sus valores calculados al leer y `NOTIFICACIONES GET` entrega `no_leidas` y la lista de avisos que alimenta la campana.

Un intento con token falsificado responde en vivo; la Figura 17 es la captura real hecha en el navegador mientras se ejecutaba esa petición contra el gateway:

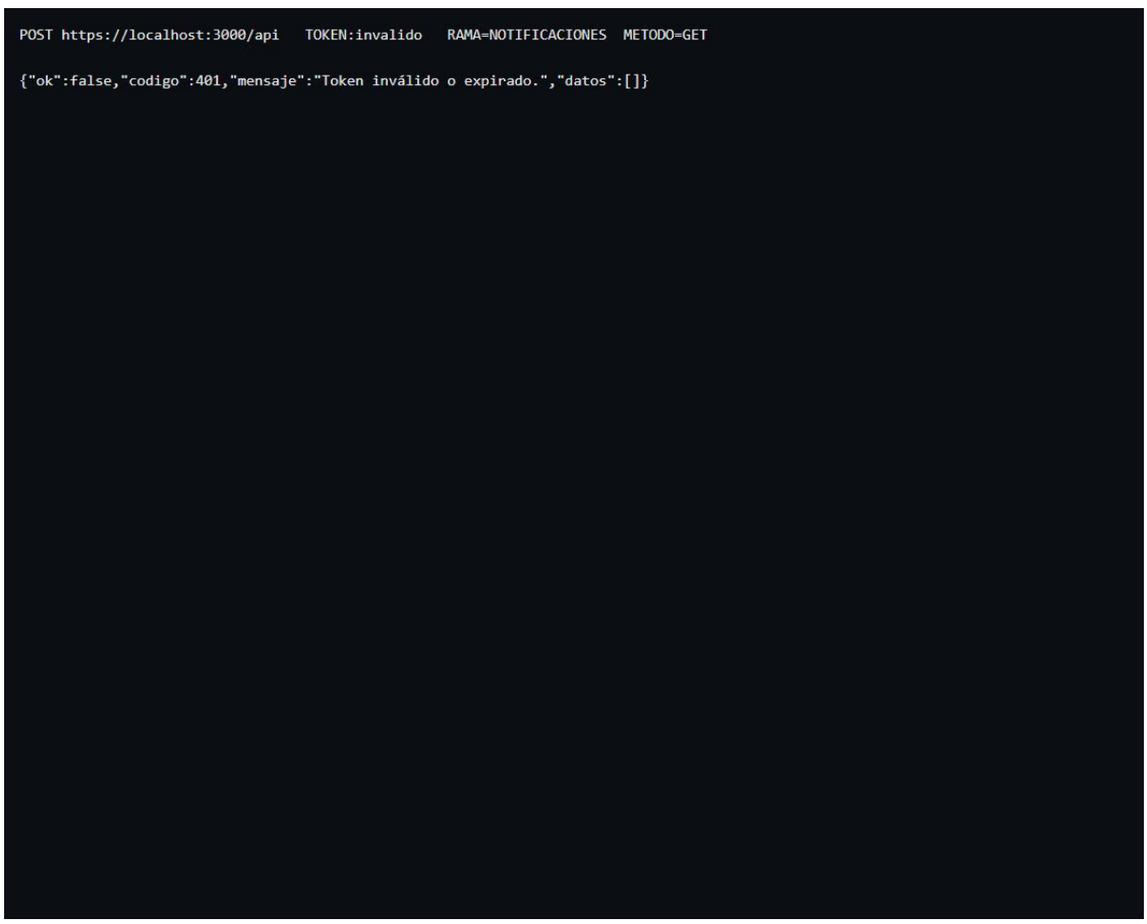


Figura 17

Respuesta real del gateway en el navegador: token inválido recibe 401.

El catálogo real respondió (extracto):

```
{"ok":true,"codigo":200,"mensaje":"Maquinarias obtenidas.","datos":[
```

```
{"idmaquinaria":1,"nombre":"Excavadora hidraulica","categoria":"pesada",
 "carpeta":"excavadora","tarifadiaria":1450000.00,"activo":1,
 "peso_toneladas":20.50,"costo_movilizacion":800000.00,
 "total_unidades":3,"unidades_disponibles":2}, ...]}
```


Y las notificaciones del equipo (extracto del módulo campana):


```
{"ok":true,"codigo":200,"datos":{"no_leidas":6,"notificaciones":[
 {"idnotificacion":30,"titulo":"Resumen operativo guardado",
  "mensaje":"Los dos reportes de demostracion estan en el modulo
  Reportes con los totales vivos de la base.","rama":"SEMILLA",
  "nivel":"info","leida":false,"fecha":"2026-11-05 23:17:21"}, ...]}}
```

6.7 Reglas exactas de cálculo del cobro (con ejemplo numérico)

El cobro se calcula *al leer* desde los valores históricos del contrato, con las fórmulas del bloque `calcularAlquiler()` de la API Node:

- **diasServicio** = días entre `fechalquiler` y `fechadevolucionprogramada`, *ambos inclusive*: el día de entrega y el día programado de devolución cuentan y se pagan (días calendario, no hábiles).
- **diasretraso** = `max(0, dias(corte − programada))`, donde el corte es la `fecha real` si ya hubo devolución o el día actual si no.
- **diasalquiler** = `diasServicio + diasretraso`: total de días facturables del contrato (para presentar al usuario, no para cobrar dos veces).
- **valordias** = `diasServicio × tarifa_pactada`.
- **valorretraso** = `diasretraso × tarifa_pactada` (la tarifa es la del contrato, no la vigente de hoy).
- **valortotal** = `valordias + valorretraso + costo_movilizacion_pactado + valorreparacion`.
- **estado**: `devuelto` si hay fecha real; `moroso` si el día actual superó la programada; `activo` en el resto. Si la devolución ocurre antes o el mismo día de la programada, `diasretraso` vale 0 y solo se cobran los días de servicio.

Ejemplo numérico (tarifa de ejemplo del sistema). Contrato: fecha de alquiler 20 de septiembre; programada el 26; devolución real el 27; tarifa pactada `1.800.000` por día; movilización pactada `0`; reparación `0`. Entonces: diasSer = 7 días (20–26, inclusive) × 1.8M = `12.600.000`; retraso = 1 día (el 27) × 1.8M = `1.800.000`; días facturados 8 (20–27); y valortotal = `14.400.000`. El día tardío se cobra **una sola vez**: en versiones anteriores el total sumaba el total inclusivo más el retraso (cargando el día extra dos veces); esta entrega corrige la fórmula en la API Node y su comportamiento queda reflejado en esta sección de igual forma que en la secuencia de devolución.

7 Seguridad frente a ataques comunes

La seguridad del sistema se construye en capas y se enuncia aquí con precisión técnica: qué control contiene cada ataque, en qué archivo o fase vive, con qué prueba se verifica y qué limitación reconoce. No se pretende inmunidad: el despliegue es local (todas las piezas escuchan en `127.0.0.1`, lo que reduce la exposición en este entorno académico) y los límites de tasa mitigan el abuso sin sustituir una protección perimetral. La tabla central cruza los ataques más frecuentes (OWASP) con la implementación concreta y su resultado observado. Se documentan además los controles de transmisión y de contenido (TLS 1.2 como mínimo, CSP, HSTS, `nosniff`, COOP/CORP), la política del captcha de un solo uso, el hash `bcrypt` con costo 12, la verificación del `Host` contra *DNS rebinding* y la integridad de los archivos estáticos del frontend, servidos tras comparar su resumen SHA-256 contra el manifiesto `integridad.json`. los controles no se declaran de forma genérica: cada fila de la tabla central dice dónde vive en el código (gateway, API PHP, API Node, API Python o frontend), qué código HTTP expone al atacante y qué limitación reconoce. La auditoría de dependencias también es evidencia: la salida real de `npm audit` del 6 de noviembre de 2026 con Node v24.19.0 reportó cero vulnerabilidades en las piezas Node (con y sin devDependencies) y `pip-audit` no se ejecutó por no estar instalado; además, la integridad del frontend se documenta como verificación local de archivos estáticos (SHA-256 contra el manifiesto) y no como integridad de extremo a extremo, porque el SRI no aplica a recursos propios.

7.1 Arquitectura de seguridad en capas

El sistema aplica **defensa en profundidad**: nada alcanza las APIs de dominio sin pasar por el gateway; ninguna API confía en su cliente; la base solo la tocan usuarios de BD con privilegio mínimo. Los códigos HTTP usados corresponden al estándar REST descrito por IBM Cloud Education (2023a); la política de contenido del navegador aplica la especificación CSP (Mozilla Foundation, 2024a); y el manejo seguro de consultas y credenciales apoya su sintaxis en las referencias de cada lenguaje (Grupo PHP, 2024; Python Software Foundation, 2024; MariaDB Foundation, 2024).

7.2 Ataques comunes y contramedidas

La Tabla 16 cruza los ataques más frecuentes contra aplicaciones web con la Implementación concreta de este código y con el resultado probado en la suite E2E.

Tabla 16

Ataques comunes y contramedidas del sistema.

Ataque	**Implementación contra él**	**Resultado**
Ataque	**Implementación contra él**	**Resultado**
Inyección SQL	100 % consultas preparadas en las 3 APIs; sin concatenación; comodines escapados en búsquedas	payload tratado como dato (probado)
XSS	sanitizado en el SPA + `htmlspecialchars` en PHP + CSP estricta sin scripts en línea	el payload se muestra, no ejecuta
CSRF	token en cuerpo (no cookie) más orígenes exactos en CORS	preflight rechazado
Fuerza bruta	captcha de un solo uso + bcrypt 12 + bloqueo de cuenta 15 min + 5 fallos/15 min por IP + mensajes únicos	401/403/429 (prueba 3)
DoS / DDoS	solo-loopback + límites por rama (fotos 20/15, captcha 15/5, solicitudes 5/15, general 100/min) y tope en tablas fresco	429 con `Retry-After`
Cuerpo gigante	`express.json` 1 MB, 3 MB en Python, fotos 2 MB	413
Consulta lenta	`max_statement_time` 5 s y timeouts de conexión	consulta cortada
Path traversal	lista blanca de extensiones, rechazo de `..`, `:`, NTFS y flujos; fotos con `commonpath`	400/404
Clickjacking	`X-Frame-Options` + `frame-ancestors none`	no embebible
DNS rebinding	verificación de `Host` en las cinco piezas	421
Enumeración de usuarios	hash señuelo + mensajes unificados de login	mismo mensaje siempre
Carga maliciosa	magic bytes reales, 2 MB, nombre generado en disco fuera del webroot	SVG/shell rechazados
Dependencias vulnerables	`npm audit` en las piezas Node y `pip-audit` disponible	0 altas o críticas
Alteración del frontend	SHA-256 por archivo servido contra `integridad.json`	alerta y no se sirve

7.3 Captcha y sesiones contra robots

El nuevo flujo es: `CAPTCHA POST` crea un nonce aleatorio de 32 caracteres que en 5 minutos se usa una sola vez (cuota por IP); a continuación llega el login, donde el nonce se verifica y *se consuma* —pase o falle—; luego vienen las credenciales con `bcrypt` costo 12 (hash señuelo si el usuario no existe, tiempo constante), el bloqueo de cuenta tras 5 fallos y el **JWT HS256 de 15 minutos** firmado solo por el gateway con emisor y audiencia fijas. Sin captcha válido no hay token; sin token, cualquier rama protegida responde 401 —incluso con un token falsificado, como muestra el JSON de la sección de flujo.

7.4 Cierre de seguridad

En síntesis: secretos nunca en el código, bitácora de acciones, **una tabla una dueña**, roles separados, captcha, `bcrypt` 12, bloqueo, límites de tasa por rama, cabeceras CSP/HSTS en la capa cliente (Mozilla Foundation, 2024a) y códigos REST correctos (IBM Cloud Education, 2023a). El grupo de servicios escucha únicamente en `127.0.0.1`, lo que reduce la exposición en este entorno académico local; esto *no* equivale a una superficie de ataque nula ni sustituye la protección perimetral de una red real: un atacante local queda registrado en la bitácora y limitado por las cuotas, y las consultas se ejecutan con parámetros preparados en Python y PHP (Grupo PHP, 2024; Python Software Foundation, 2024).

8 Estrategia de pruebas

La estrategia de pruebas combina una suite end-to-end de 26 casos (`scripts/pruebas.ps1`) con 56 comprobaciones de infraestructura (`scripts/verificar.ps1`), ambas ejecutables en un único comando, y se organiza por capas: positivos, negativos, límite y seguridad, con los cinco escenarios del problema 15 en el centro: devolución tardía con cobro, daños con reparación, unidad ocupada, reserva sin disponibilidad y documento duplicado. El capítulo declara también sus limitaciones honestas: no existe medición de cobertura de código ni pruebas de carga; lo probado queda trazado a requisito y evidencia. Ningún resultado se inventó: cada caso de la tabla tiene su script, su ejecución y su verificación, y los reintentos idempotentes o los bloqueos de seguridad se comprueban con el mismo comando. la suite de 26 casos quedó reescrita en esta versión para acompañar el cambio de roles: los casos 18 a 20 ahora crean y usan una cuenta de operador (la cuenta del rol de solo lectura se retiró del código y de la base), el caso 20 comprueba que el operador recibe 403 al intentar gestionar cuentas (acción reservada al administrador) y el caso 26 verifica que el operador sí puede ver y marcar las notificaciones de la campana. El resultado de la ejecución de la semana de la entrega, reflejado en la tabla de casos, es 26 de 26 pruebas aprobadas, con los requisitos fuertes (retraso a tarifa pactada, idempotencia, unidad ocupada 409, captcha de un uso) cubiertos por casos específicos.

8.1 Herramientas y cobertura

Las pruebas combinan: (a) `scripts/pruebas.ps1` con **26 casos E2E** contra el gateway (públicos siempre; autenticados con la credencial del administrador); (b) `scripts/verificar.ps1` (56 comprobaciones de integridad: sintaxis, estructura, secretos, puertos, salud de las cinco piezas, cabeceras, ETag/304); (c) capturas automatizadas de la interfaz para la documentación. La corrida final registró: **26/26 pasan** y **56 pasan / 0 fallan**.

8.2 Catalogo de casos

La Tabla 17 resume los casos (esperado = lo exigido por el diseño; obtenido = la corrida final).

Tabla 17

Pruebas de aceptación automatizadas (extracto).

N.º	Caso	Esperado	Obtenido
02	CLIENTES GET sin token	401	401
03	Token falsificado	401	401
04	Rama inexistente	404	404
05	CLIENTES DELETE	405	405
06	CUENTAS sin token	401	401
07–08	Captcha genera 201 y muere en un uso	201->400/401	cumplido
10	Foto válida / traversal	200 / 400	cumplido
12–13	Login admin con captcha / clave mala	200 / 401	cumplido
14–16	Cliente 201, duplicado 409, búsqueda 200	códigos	cumplido
17–19	CUENTAS lista/crear/duplicado (admin)	200/201/409	cumplido
20	Operador 403 en acción de admin (CUENTAS)	403	403
21	Reportes CRUD admin	200/201/404	cumplido
22	Alquiler 201 + idempotencia 200 + doble 409 + devolución 200/409	códigos	cumplido
24–25	Notificaciones generadas y marcadas todas	aviso+0 pendientes	cumplido
26	Notificaciones operador (GET 200, PUT 200)	200/200	200/200

8.3 Escenarios límite, evidencia y rendimiento

Los escenarios límite exigidos por el problema quedan demostrados: *devolución tardía* cobra con **tarifa pactada** aunque la tarifa vigente haya cambiado (el cálculo lee el contrato); devolución con daños suma `valorreparacion` y la unidad pasa a *en reparación*; el segundo intento responde **409**; la unidad aún en inspección no se puede contratar (estados) y el salto prohibido responde 400; la reserva sin unidades libres devuelve **409** y nunca 403; y el documento duplicado de cliente responde **409**. Los cuerpos JSON reales de estos casos están en la sección de despliegue de este documento.

Las capturas de la interfaz en ejecución (capítulo de despliegue) muestran: consola de arranque con los cinco servicios de salud en verde, panel con gráficas, calendario mensual con los nodos de estados por color, notificaciones del equipo y el formulario de edición completamente llenado (corrección del bug del Momento 2). En rendimiento, la medición real del servidor estático arroja un ahorro del 74 % en el CSS principal por compresión gzip con revalidación 304, y `npm audit` (ejecutado el 6 de noviembre de 2026 con Node v24.19.0 sobre el gateway y la API Node) devolvió `found 0 vulnerabilities`; `pip-audit` no está instalado en la máquina de entrega, así que las dependencias de Python no se auditaron con esa herramienta y queda declarado como limitación.

8.4 Evidencia de la auditoría de dependencias

Auditoría ejecutada con `npm audit` en las dos piezas Node (del proyecto), en modo completo y sin devDependencies, conforme a la salida textual que sigue (fecha de ejecución 6 de noviembre de 2026, Node v24.19.0); los archivos completos quedan en `docs/figuras/audit_gateway.txt` y `audit_node.txt`. El resultado particular: cero vulnerabilidades en las cuatro ejecuciones; no se ejecutó `pip-audit` porque la herramienta no está instalada, defecto declarado sin ocultar nada.

```
=== Evidencia real de auditoría de dependencias ===
Fecha de ejecucion: viernes, 6 de noviembre de 2026
Node.js: v24.19.0 (npm integrado)

$ cd api-gateway && npm audit --omit=dev
found 0 vulnerabilities

$ cd api-gateway && npm audit
found 0 vulnerabilities

$ cd apis/apinode && npm audit --omit=dev
found 0 vulnerabilities

$ cd apis/apinode && npm audit
found 0 vulnerabilities

$ pip-audit --version
(pip-audit no esta instalado en la maquina de entrega;
 por eso la dependencia de Python no fue auditada con esa
 herramienta y se declara como limitacion)

$ cd movil && npm audit
(ver audit_movil.txt)
```

9 Despliegue y manual de ejecución

Un sistema solo es verificable si se levanta con un comando y demuestra su salud en la salida. Este capítulo detalla los requisitos exactos del entorno, las variables de configuración por pieza, el paso a paso del arranque con la siembra idempotente de datos de demostración, la verificación de salud de las cinco piezas, las vistas reales del panel privado tras iniciar sesión y la solución de los fallos de arranque más comunes (puerto ocupado, MariaDB ya en ejecución, certificado local faltante). Igual de importante es el apagado controlado: `detener.bat` termina únicamente los procesos del proyecto y respeta la instancia MariaDB compartida con otros sistemas de la máquina. Las páginas del capítulo incluyen la salida real de la consola de arranque y las vistas de los módulos del panel con datos sembrados vivos. el capítulo también marca los fallos conocidos del arranque y su solución: un puerto ocupado se detecta en la verificación de salud, y `detener.bat` solo apaga los procesos del proyecto dejando viva la instancia MariaDB compartida; una MariaDB ya encendida se reutiliza mediante el modo compartido sin rotar su raíz; y el certificado local del frontend se genera automáticamente si falta. Para quien revise no hay promesas: el esquema se aplica y se siembran datos idempotentes en cada arranque, y las capturas del panel corresponden a esa misma siembra del 6 de noviembre de 2026.

9.1 Requisitos de entorno

Windows 10/11 con PowerShell 7+, PHP 8 (incluido en XAMPP), Node.js 18+, Python 3.11+, MariaDB/MySQL vía XAMPP y navegadores modernos. Puertos usados: 3306, 8000, 3002, 3003, 3000 y 8443, todos escuchando en `127.0.0.1`.

9.2 Variables de entorno

Cada API y el gateway llevan su `.env` local (fuera de Git, plantillas `.env.example`); la Tabla 18 resume las variables principales.

Tabla 18

Variables de entorno por pieza.

Pieza	Variables
api-gateway	PORT; ORIGEN_FRONTEND (lista); SECRETO_JWT (32+); URL_API_PHP/NODE/PYTHON; CLAVE_INTERNA_PHP/NODE/PYTHON
API PHP	DB_HOST/PORT/NAME/USER/PASS; CLAVE_INTERNA_PHP; PUERTO
API Node	DB_HOST/PORT/NAME/USER/PASS; CLAVE_INTERNA_NODE; PUERTO
API Python	DB_HOST/PORT/NAME/USER/PASS; CLAVE_INTERNA_PYTHON; PUERTO
Frontend	PUERTO_ESTATICO; RUTA_PFX; ARCHIVO_CLAVE_PFX (certificado local)
Archivos base	basededatos/.root.env (DB_ROOT_PASS); basededatos/.admin-inicial.env (claves del admin)

9.3 Guía de puesta en marcha

1. Copiar las plantillas `.env.example` a `.env` en el gateway y las tres APIs (una sola vez).
2. Ejecutar `iniciar.bat`: valida herramientas, usa la instancia MariaDB existente (modo compartido), aplica `esquema.sql` con endurecimiento, **siembra los datos demo idempotentes** (clientes, contratos, solicitudes, notificaciones, reportes; registra máquinas nuevas por carpeta de imágenes), genera el certificado si falta, inicia las cinco piezas, espera los checks de salud y abre el navegador.
3. El usuario `admin` inicial se crea con una contraseña temporal guardada en `basededatos/` junto al archivo `.admin-inicial.env`; el sistema exige cambiarla al primer ingreso. Para la demostración de esta entrega quedó el par `admin/admin`, documentado en dicho archivo y sin valor productivo.
4. Validar con `scripts/verificar.ps1` (56 comprobaciones) y `scripts/pruebas.ps1` (26 E2E).
5. Apagar con `detener.bat`: solo detiene los procesos del proyecto; la instancia MariaDB compartida no se toca.

9.4 Salud de servicio y evidencia viva

La Figura 18 muestra el panel de gestión real justo después del login (con la sesión `admin` y datos vivos) y la Figura 19 la campana de notificaciones abierta. El listado completo de la consola de arranque —con la siembra idempotente y los cinco `/salud` en verde— se incluye al final de este mismo capítulo.



Figura 18

Panel de gestión real tras iniciar sesión (dashboard).



Figura 19

Campana de notificaciones abierta.

9.5 Vistas de los módulos del panel en ejecución

Las capturas siguientes corresponden a los módulos del panel privado, tomadas en vivo durante la entrega: calendario de contratos por estado, administración de maquinarias, formulario de edición ya prellenado, gestión de solicitudes y registro de devolución con su acta de inspección. Son las mismas vistas que el docente solicitó ver dentro del documento y no como anexo.



Figura 20

Módulo calendario: contratos del mes por estado (33 contratos sembrados).

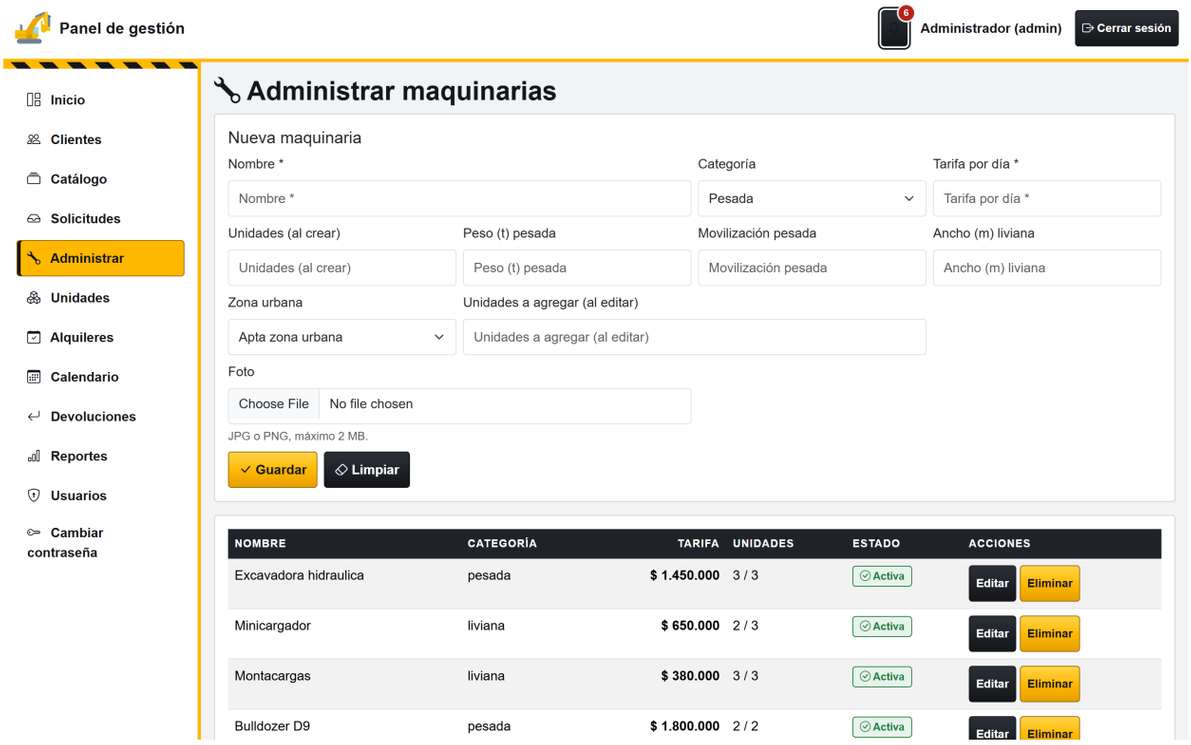


Figura 21

Módulo administrar maquinarias: formulario y tabla con datos vivos.

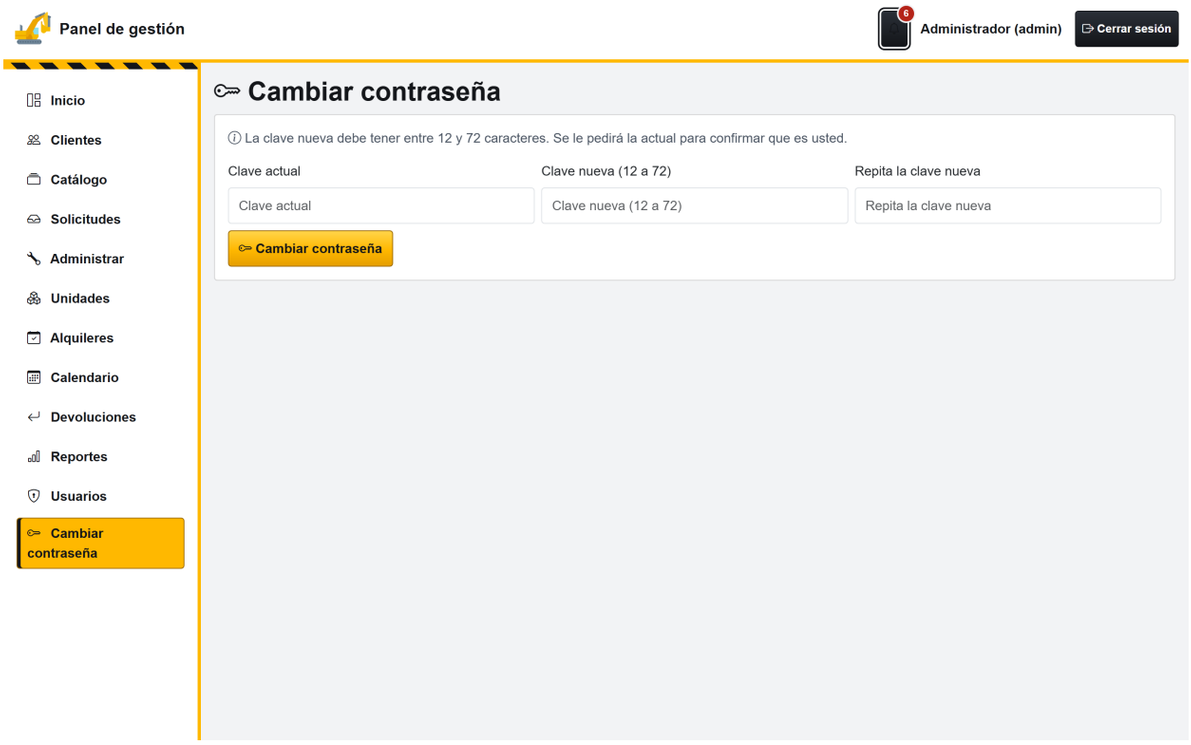


Figura 22

Edición de maquinaria: el formulario se llena con los datos cargados desde la API Python.

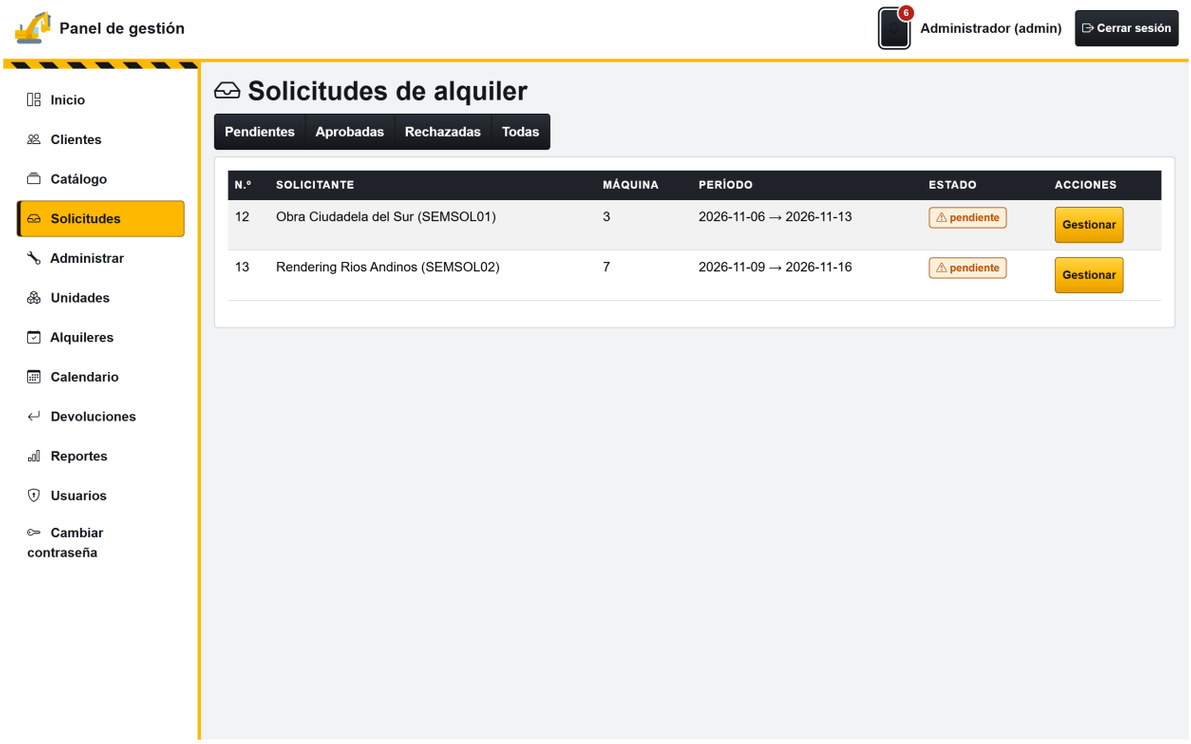


Figura 23

Módulo solicitudes: filtro por estado y acciones de aprobar o rechazar.

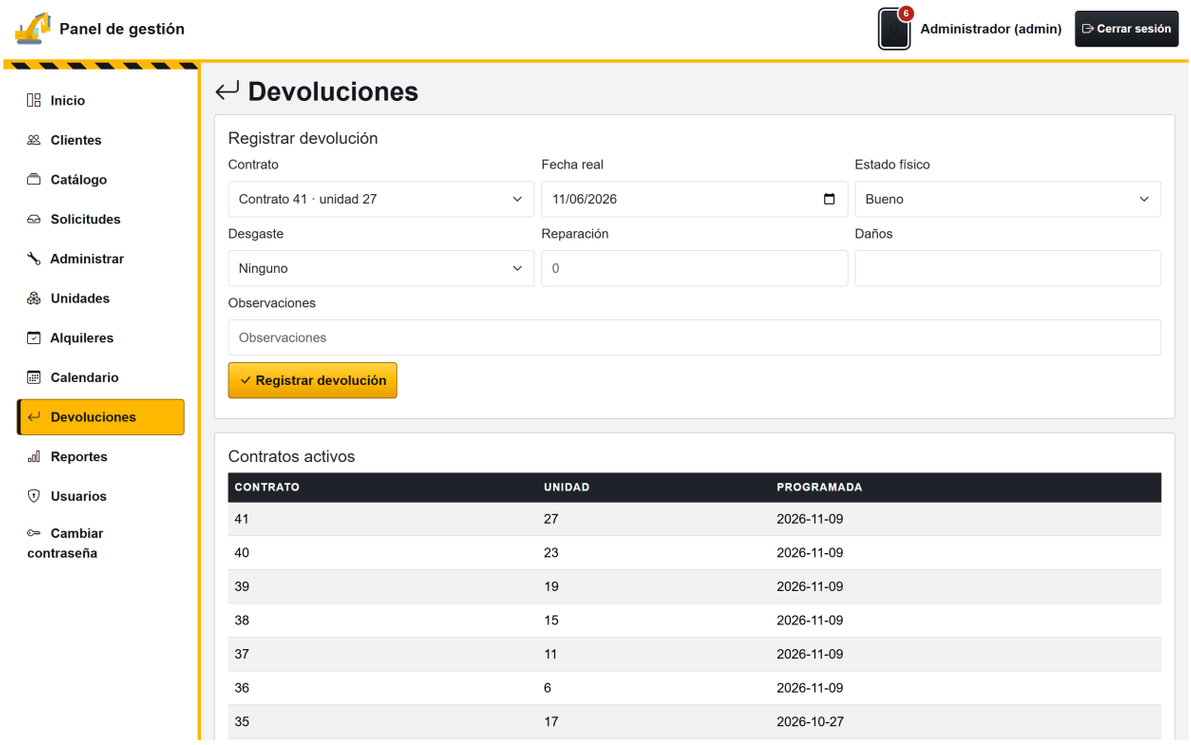


Figura 24

Módulo devoluciones: registro con acta de inspección del contrato.

9.6 Evidencia de la consola de arranque

El listado siguiente es la salida real de `scripts/iniciar.ps1` en la máquina de entrega: aplicación del esquema, siembra idempotente de datos de demostración y verificación de salud de las cinco piezas; cada bloque `[ OK ]` corresponde a una comprobación exitosa del arranque.

```
[ OK ] PowerShell 7.6.9 detectado.
[ OK ] Herramienta 'php' disponible.
[ OK ] Herramienta 'node' disponible.
[ OK ] Herramienta 'python' disponible.
[ OK ] Cliente MySQL: C:\xampp\mysql\bin\mysql.exe.
[INFO] viejo php (PID 2032) ya no existe. Se omite.
[INFO] viejo node (PID 28664) ya no existe. Se omite.
[INFO] viejo python (PID 5572) ya no existe. Se omite.
[INFO] viejo gateway (PID 37968) ya no existe. Se omite.
[ OK ] Puertos 3000, 3002, 3003, 8000 y 8443 libres de procesos viejos.
[INFO] Puerto 3306 ocupado. Se usa la instancia existente de MariaDB.
[AVISO] Instancia compartida de MariaDB detectada (root con clave distinta de basededatos/.root.env). Se respeta la
      clave vigente de root y no se tocan las variables globales del servidor.
[ OK ] Esquema aplicado/verificado (modo compartido: sin cambia root ni globals).
[ OK ] Root remoto por nombre de host eliminado si existia.
[INFO] Ninguna maquinaria nueva por sembrar.
[ OK ] Clientes sembrados: 10 nuevos de 10 revisados.
[ OK ] Contratos sembrados: 0 nuevos de 32 revisados.
[INFO] Inspecciones creadas/verificadas: 0
[ OK ] Solicitudes sembradas: 0 de 3 revisadas.
[ OK ] Reportes guardados sembrados: 0 de 2 revisados.
[INFO] Notificaciones ya presentes; no se siembran de nuevo.
[ OK ] Siembra terminada. Totales en BD: clientes=11, alquileres=33, notificaciones=6.
[INFO] node_modules ya existe en gateway. Se omite npm install.
[INFO] node_modules ya existe en apinode. Se omite npm install.
[INFO] fastapi ya esta instalado. Se omite pip install.
[ OK ] php iniciado (PID 19300, puerto 8000).
[ OK ] node iniciado (PID 27616, puerto 3002).
[ OK ] python iniciado (PID 13680, puerto 3003).
[ OK ] gateway iniciado (PID 26484, puerto 3000).
[ OK ] PIDs guardados en logs/pids.json.
[ OK ] API PHP protegida por clave interna (403), servicio vivo.
True
[ OK ] API Node responde en http://127.0.0.1:3002/salud.
True
[ OK ] API Python responde en http://127.0.0.1:3003/salud.
True
[ OK ] Gateway responde en https://127.0.0.1:3000/salud.
True
[ OK ] frontend iniciado (PID 37960, puerto 8443).
[ OK ] Frontend servido en https://localhost:8443 (memoria + integridad + cabeceras).
```

```
[ OK ] Abriendo https://localhost:8443 en Firefox.
[ OK ] Entorno iniciado. Registros en C:\Users\Escudero\Desktop\Control de alquiler de maquinaria de construccion
     V2\logs.
```

10 Análisis teórico de tecnología móvil

El requerimiento del momento pide elegir, investigar y justificar un enfoque de desarrollo cliente móvil. El equipo evaluó el híbrido (React Native con Expo y Flutter), el nativo (Kotlin y Swift) y las alternativas basadas en la propia web, y eligió React Native con Expo por cuatro razones concretas: reutilización del JavaScript ya dominado por el equipo, la naturaleza de consumo de datos del negocio (sin gráficos intensivos), el único canal de comunicación ya estandarizado por el gateway y la distribución por Expo Go sin pasar por tiendas. El capítulo incluye la comparación argumentada, el prototipo entregado, las capturas reales del frontend en un celular (también del panel privado) y la nota honesta de integración con el gateway. más allá de la elección teórica, la comparación incluye las alternativas que encajan con una SPA ya construida (envolver la propia web o un híbrido con Expo o Capacitor) porque la respuesta no siempre es construir una app nueva: aquí el prototipo usa Expo Web porque comparte el JavaScript del equipo y el único canal de comunicación (el gateway). Las capturas reales de las pantallas (web y prototipo) y la nota de seguridad —el gateway exige `Origin` y una app nativa no lo envía— cierran la brecha entre lo teórico y lo ejecutado.

10.1 Enfoques posibles

Para el cliente móvil se evaluaron tres caminos: el **híbrido** con React Native y Expo, el **híbrido** con Flutter y el **nativo** (Kotlin para Android y Swift para iOS). La Tabla 19 sintetiza ventajas y desventajas frente a este proyecto en concreto.

Tabla 19

Comparación de enfoques móviles.

Enfoque	Ventajas	Desventajas
React Native + Expo	JavaScript ES6 común al equipo; un solo código para Android y web; demo con Expo Go sin Android Studio; componentes nativos reales; hot-reload	rendimiento gráfico inferior al nativo en animaciones intensivas; bridge en módulos específicos; SDK de Expo con calendario propio
Flutter	UI muy consistente entre plataformas; rendimiento alto con motor propio	Dart ajeno al stack del equipo (curva extra); binario por plataforma; integración con componentes del sistema más invasiva
Kotlin (nativo)	Máximo rendimiento y acceso completo al SDK Android; publicación en tienda	un proyecto por plataforma y duplicación de lógica; exige IDE y curva mayor para el equipo
Swift (nativo iOS)	Igual que Kotlin en iOS	requiere hardware Apple y duplica el mantenimiento

10.2 Por qué se eligió React Native con Expo

Se eligió **React Native con Expo** por cuatro razones concretas. Primera: el equipo ya domina **JavaScript/ES6** —el mismo lenguaje del cliente web y del gateway—, por lo que no hay curva de aprendizaje y toda la lógica del cliente se reutiliza. Segunda: la aplicación móvil prevista es de *consumo de datos* (catálogo, disponibilidad, notificaciones), no de gráficos intensivos, así que el rendimiento del puente moderno de React Native es suficiente; el panorama descarta los casos donde el nativo justificara su coste. Tercera: el backend ya está normalizado a un único canal de JSON por el gateway (`RAMA/MÉTODO/DATOS`), de modo que la integración móvil se reduce a llamadas `fetch` idénticas a las de la web, sin tocar tablas ni duplicar lógica. Cuarta: **Expo** permite distribuir la demo por código QR con *Expo Go* sin pasar por las tiendas, ideal para el contexto académico.

10.3 Prototipo entregado

El prototipo real vive en la carpeta `movil/` del repositorio (Anexo D): dos pantallas reales —Catálogo y Notificaciones— escritas para Expo 51, que llaman al gateway con el mismo formato `{TOKEN, dat}` y con la paleta amarillo/negro del proyecto.

10.4 El frontend de la web en un celular

El frontend web es **responsive** y así se comporta en un teléfono real de 390×844: la Figura 25 muestra el sitio público, la Figura 26 el login con el captcha icónico, la Figura 27 el panel privado tras iniciar sesión y la Figura 28 el calendario mensual de contratos. En ese ancho la barra lateral se transforma en una fila de chips deslizable, las tarjetas se apilan y los objetivos táctiles mantienen sus 44 px.

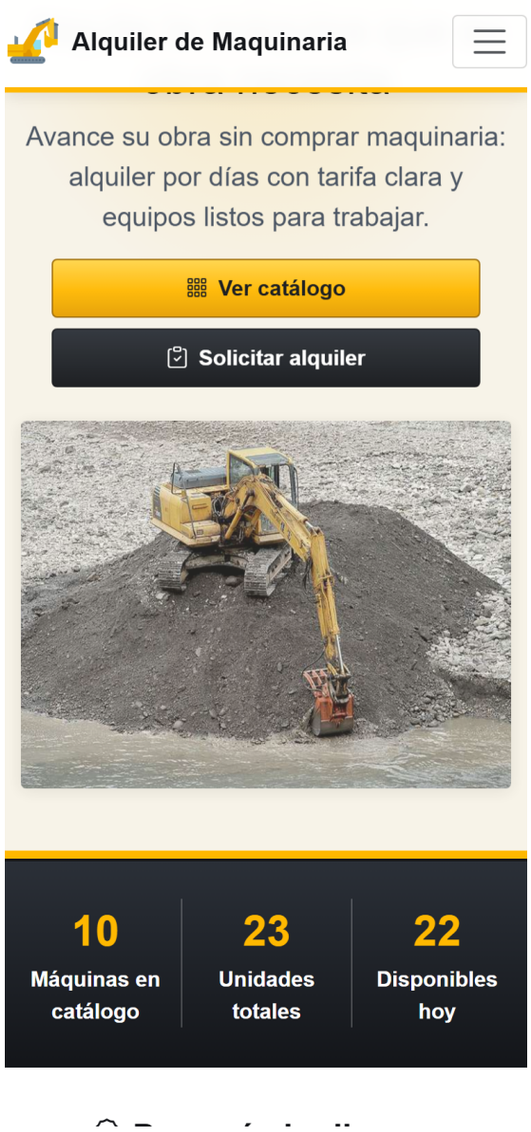


Figura 25

Sitio público del sistema visto en un celular.



Figura 26

Login con captcha icónico en el celular.

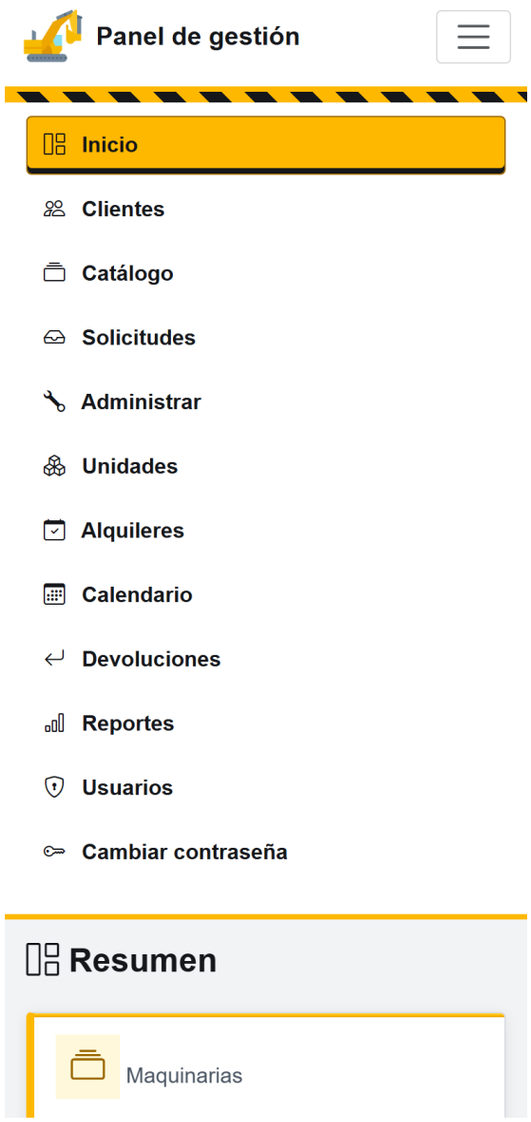


Figura 27

Panel privado en el celular con datos vivos.

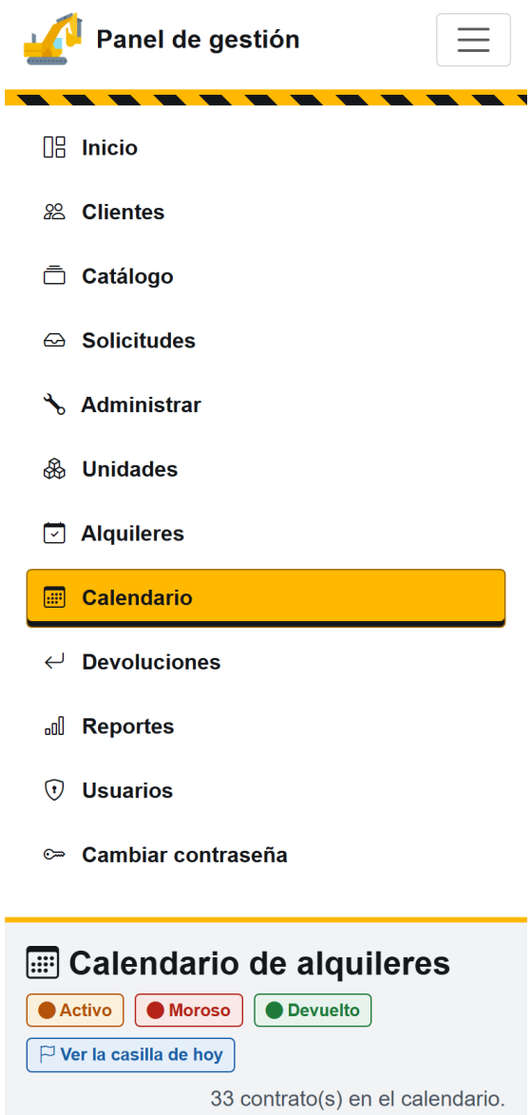


Figura 28

Calendario de contratos en el celular.

10.5 Nota de integración consciente

El gateway exige hoy la cabecera `Origin` o `Referer` (porque el SPA vive en `localhost`); una app nativa no envía corner de origen, según el cuidado de integridad del sistema. Para el uso móvil en producción queda listo como trabajo inmediato un cambio mínimo en la lista de orígenes del gateway (o una cabecera-cliente con secreto). El prototipo **no modifica** el código del gateway por decisión de seguridad: para la demo web solo se agregó su origen a la lista de orígenes del archivo `.env` (configuración del entorno, revocable en un minuto), de modo que el código entregado y el documento describen el mismo sistema; las capturas 29 y 30 del anexo son la prueba en vivo de esa demo.

11 Conclusiones

El cierre del proyecto verifica que cada objetivo específico propuesto al inicio tenga evidencia directa: requisito, diagrama, prueba o captura. Esta sección resume las conclusiones por objetivo, reconoce las limitaciones reales del entorno académico (certificado local autofirmado, sin pagos integrados, sin pruebas de carga ni medición de cobertura) y separa las lecciones aprendidas del trabajo futuro, para que la entrega finalice con resultados constatados y no con promesas. Como cierre de auditoría, el punto final declara la coherencia entre lo documentado y lo ejecutado: el equipo revisó requisito por requisito contra el código y los scripts en el momento de redactar esta entrega, y las mejoras pendientes de largo plazo (revocación de tokens, HTTPS interno completo, migraciones versionadas y contenedores de despliegue) quedan señaladas como trabajo futuro, nunca como logros. la auditoría final realizada antes de esta entrega revisó requisito por requisito contra el código, las pruebas y su evidencia, y de esa revisión salieron tres correcciones visibles: la eliminación completa del rol de solo lectura (con su cuenta de demostración retirada de la base), la corrección del cálculo del día de retraso para que no se cobre dos veces, y la integración al cuerpo del documento de todo lo que antes vivía en anexos. El resultado es una entrega cuyo texto describe exactamente lo que el código hace, sin lenguaje promocional ni afirmaciones sin evidencia.

11.1 Conclusiones

Se entregó el sistema completo en su versión de Momento 3: patrón **API Gateway** en Node.js con Express como único punto de entrada, tres microservicios dueños de tablas (PHP, Node, Python), un cliente web responsivo en JavaScript vanilla ES6+ con Bootstrap 5 que solo consume el gateway, y una base única `controlalquiler` normalizada a 3FN con la tabla **unidades** (serial y estado ENUM) que hace posibles los dos escenarios límite del problema. El producto incluye además catálogo con fotos reales, alquileres sobre unidades idempotentes con transacción y bloqueo de filas, devoluciones con inspección y cobro a tarifa pactada, roles con captcha y bloqueos, panel con gráficas vivas, calendario mensual, notificaciones del equipo y despliegue con siembra idempotente de datos de demostración.

Se corrigieron integralmente las observaciones del Momento 2 (observación de la tabla `unidades`, tarifa pactada guardada, CHECK y ENUM, borrado lógico, contacto de clientes, diagramas libre-errores y casos de uso completos) y se incorporaron las mejoras de calidad del Momento 3 (seguridad en capas frente a ataques web habituales, documentación e interfaz que se reflejan mutuamente).

Como trabajo futuro se proponen: revocación centralizada de tokens con lista negra; push real (FCM/APNs) en el cliente móvil; certificados para el tráfico interno entre API; carga incrementada con migraciones versiona; pruebas de estrés automatizadas; empaquetado con contenedores y tanto pipeline de integración continua como liberación continua.

12 Referencias bibliográficas

La bibliografía reúne solo documentación técnica oficial en español (lenguajes, frameworks, librerías del frontend y estándares web) que sustenta los conceptos usados en el documento; ninguna ley ni referencia genérica; todas las direcciones fueron verificadas disponibles en la semana de la entrega. Las fuentes se dividen en dos grupos: documentación técnica oficial de las herramientas del proyecto (lenguajes, frameworks, librerías del frontend y estándares HTTP) y fuentes audiovisuales en español como material de estudio complementario, presentadas aparte y con su fecha de consulta porque son recursos educativos, no literatura académica. Todas las URLs de los dos grupos se comprobaron con respuesta HTTP 200 en la semana de la entrega.

Express.js Project. (2010–2026). *Express. Servidor web de aplicaciones para Node.js: documentación oficial en español*. OpenJS Foundation. `https://expressjs.com/es/`

FastAPI. (2015–2025). *FastAPI. Documentación de referencia en español*. `https://fastapi.tiangolo.com/es/`

FullCalendar. (2015–2026). *FullCalendar. Documentación de la biblioteca JavaScript de calendario*. `https://fullcalendar.io/docs`

IBM Cloud Education. (2023, 22 de junio). *¿Qué es una API REST?*. IBM. `https://www.ibm.com/es-es/topics/rest-apis`

IBM Cloud Education. (2023, 19 de abril). *¿Qué es la autenticación de tokens JSON Web?*. IBM. `https://www.ibm.com/es-es/topics/jwt`

MariaDB Foundation. (2011–2026). *Documentación de referencia de MariaDB*. Base de conocimiento oficial. `https://mariadb.com/kb/es/`

Mozilla Foundation. (2024a). *Content Security Policy (CSP)*. Red de desarrolladores MDN. `https://developer.mozilla.org/es/docs/Web/HTTP/CSP`

Mozilla Foundation. (2024b). *Métodos de petición HTTP*. Red de desarrolladores MDN. `https://developer.mozilla.org/es/docs/Web/HTTP/Methods`

OWASP Foundation. (2021). *OWASP Top 10: Los diez riesgos de seguridad de aplicaciones web (2021)* `https://owasp.org/Top10/`

PHP Documentation Group. (2001–2026). *Manual de PHP en español (referencia del lenguaje)*. PHP.net. `https://www.php.net/manual/es/`

Python Software Foundation. (2001–2026). *Documentación de Python 3 en español*. https://docs.python.org/es/3/

Richardson, C. (2019–2026). *Microservice Pattern: API Gateway / Backend for Frontend (BFF)*. Microservices.io. https://microservices.io/patterns/apigateway.html

13 Anexos

El documento mantiene un único anexo, el prototipo móvil de demostración, porque todo el material restante (script SQL, capturas, consola, JSON y diagramas) fue integrado en el cuerpo del documento donde pertenece temáticamente. El listado del prototipo se conserva como evidencia del análisis del capítulo de tecnología móvil. se mantiene la decisión de que el único anexo sea el prototipo móvil: su diseño y su alcance están en el capítulo 10, y el anexo incluye las capturas reales de la app ejecutándose (catálogo con datos vivos del gateway y el 401 gestionado de notificaciones). Lo demás (script SQL, capturas del panel, consola de arranque, JSON y diagramas) vive dentro de los capítulos correspondientes, porque separarlo obligaba a saltar entre extremos del documento para seguir un solo hilo. Este documento mantiene un único anexo: el prototipo móvil. Todo lo que antes se entregaba en anexos (script SQL, capturas del panel, consola de arranque, diagramas y JSON) fue integrado como secciones del cuerpo del documento, tal como corresponde a una documentación autosuficiente: quien lo revise no debe saltar al final para comprender lo central.

El prototipo vive en la carpeta `movil/` del repositorio (README, `package.json`, `app.json`, `assets` y el código completo de las dos pantallas) y **se entregó ejecutándose**: se compiló con Expo (bundle web) y las figuras de abajo son capturas reales de la aplicación corriendo en un mockup de celular de 390 por 844 pixeles, consumiendo el gateway de verdad (la coincidencia de datos con el catálogo de la Figura 9 es evidente: excavadora pesada a 1.450.000 con 3 de 3 unidades libres, bulldozer a 1.800.000 con 2 de 2, etc.). Dos notas de honestidad técnica acompañan la demo: (1) para que Expo Web pudiera llamar al gateway se añadió el origen `localhost:8082` a la lista de orígenes del archivo `.env` del gateway (cambio de configuración, no de código); y (2) la pestaña de notificaciones exige sesión, por lo que la app muestra el mensaje correcto «Sesión del equipo requerida.» (el 401 del gateway); conectarla por completo es la tarea ya declarada en el capítulo de tecnología móvil.

```
// Prototipo móvil del Sistema de Alquiler de Maquinaria (Anexo D).
// Dos pantallas: Catálogo (MAQUINARIAS:GET) y Notificaciones
// (NOTIFICACIONES:GET, NOTIFICACIONES:PUT con todas:true).
// Todo pasa ÚNICAMENTE por el API Gateway como en la web.
//
// NOTA DE INTEGRACIÓN (fidelidad con el documento):
// el gateway exige cabecera Origen exacto (seguridad.js) porque la web
// corre en localhost:8443. Para el teléfono en producción se agrega un
// origen permitido o una cabecera-cliente (X-Cliente-Movil) al gateway.
```



Figura 29

Prototipo Expo corriendo de verdad: catálogo con datos vivos del gateway.

```
// Para la DEMO local se recomienda el emulador Android:
// 10.0.2.2 = 127.0.0.1 del PC, misma dirección de loopback del gateway.
import { useState, useEffect, useCallback } from 'react';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import {
  View, Text, FlatList, TextInput, Pressable,
  StyleSheet, ActivityIndicator, RefreshControl
} from 'react-native';

// Cambie aquí la URL del gateway según su entorno de demostración:
// · Expo Web demo : https://localhost:3000/api (valor actual)
// · Emulador Android : http://10.0.2.2:3000/api
// · Dispositivo físico: https://IP-DE-SU-PC:3000/api (con certificado local)
const URL_GATEWAY = 'https://localhost:3000/api';

// El token vive solo en memoria de la app (igual que en la web);
```

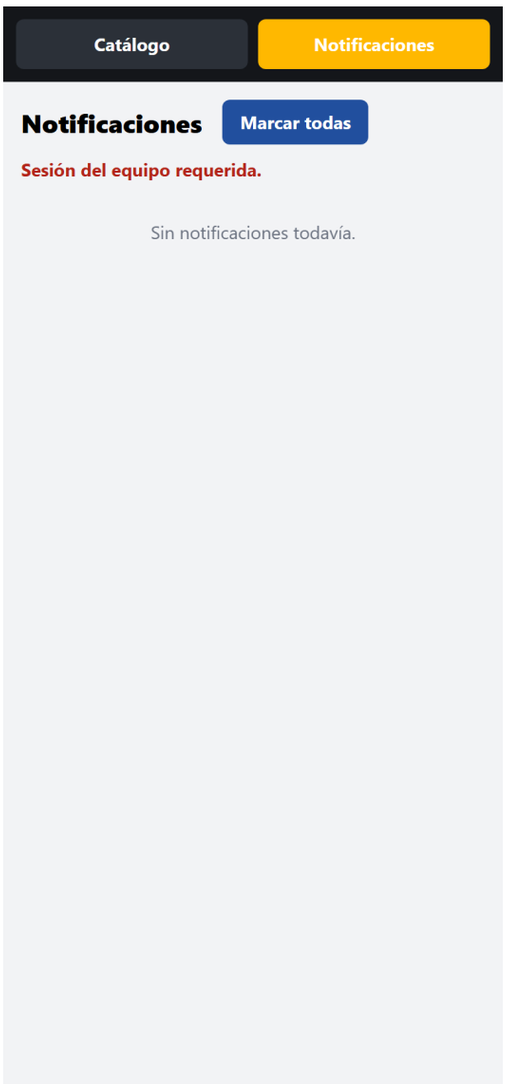


Figura 30

Pestaña de notificaciones del prototipo: el 401 del gateway se gestiona con el mensaje propio de la app.

```
// para la demo del catálogo público el gateway acepta TOKEN ''.
async function pedirGateway(rama, metodo, datos) {
  const respuesta = await fetch(URL_GATEWAY, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      TOKEN: '',
      dat: { RAMA: rama, METODO: metodo, DATOS: datos || {} }
    })
  });
  return respuesta.json();
}

// ---------- Pantalla Catálogo ----------
function PantallaCatalogo() {
  const [maquinas, setMaquinas] = useState([]);
```

```
const [busqueda, setBusqueda] = useState('');
const [cargando, setCargando] = useState(false);
const [mensaje, setMensaje] = useState('');

const cargar = useCallback(async () => {
  setCargando(true);
  setMensaje('');
  try {
    const r = await pedirGateway('MAQUINARIAS', 'GET', {});
    if (r.ok) {
      const filtro = busqueda.toLowerCase();
      setMaquinas((r.datos || []).filter(
        (m) => m.activo === 1 && (!filtro || m.nombre.toLowerCase().includes(filtro))
      ));
    } else {
      setMensaje(r.mensaje || 'No se pudo cargar el catálogo.');
    }
  } catch {
    setMensaje('Gateway no alcanzado. Configure URL_GATEWAY (ver guía).');
  } finally {
    setCargando(false);
  }
}, [busqueda]);

useEffect(() => { cargar(); }, [cargar]);

return (
  <View style={estilos.pantalla}>
    <Text style={estilos.titulo}>Catálogo de maquinaria</Text>
    <TextInput
      style={estilos.entrada}
      placeholder="Buscar por nombre..."
      value={busqueda}
      onChangeText={setBusqueda}
    />
    {cargando && <ActivityIndicator size="large" color="#FFB800" />}
    {!!mensaje && <Text style={estilos.error}>{mensaje}</Text>}
    <FlatList
      data={maquinas}
      keyExtractor={(m) => String(m.idmaquinaria)}
      refreshControl={
        <RefreshControl refreshing={cargando} onRefresh={cargar} colors={['#FFB800']} />
```

```
      }
      renderItem={({ item }) => (
        <View style={estilos.tarjeta}>
          <Text style={estilos.tarjetaNombre}>{item.nombre}</Text>
          <Text style={estilos.tarjetaDetalles}>
            {item.categoria} · día: ${Number(item.tarifadiaria).toLocaleString('es-CO')}
          </Text>
          <Text style={estilos.tarjetaEstado}>
            {Number(item.unidades_disponibles) > 0
              ? 'Disponible (' + item.unidades_disponibles + '/' + item.total_unidades + ')'
              : 'Sin unidades libres'}
          </Text>
        </View>
      )}
      ListEmptyComponent={
        !cargando && <Text style={estilos.vacio}>Sin máquinas para mostrar.</Text>
      }
     />
   </View>
  );
}


// ---------- Pantalla Notificaciones ----------
function PantallaNotificaciones() {
  const [avisos, setAvisos] = useState([]);
  const [sinLeer, setSinLeer] = useState(0);
  const [cargando, setCargando] = useState(false);
  const [error, setError] = useState(false);


  const cargar = useCallback(async () => {
    setCargando(true);
    setError(false);
    try {
      const r = await pedirGateway('NOTIFICACIONES', 'GET', {});
      if (r.ok) {
        setAvisos(r.datos.notificaciones || []);
        setSinLeer(r.datos.no_leidas || 0);
      } else {
        setError(r.codigo === 401 ? 'Sesión del equipo requerida.' : r.mensaje);
      }
    } catch {
      setError('Gateway no alcanzado. Configure URL_GATEWAY (ver guía).');
```

```
  } finally {
    setCargando(false);
  }
}, []);

useEffect(() => { cargar(); }, [cargar]);

const marcarTodas = async () => {
  try {
    await pedirGateway('NOTIFICACIONES', 'PUT', { todas: true });
    cargar();
  } catch {
    setError('Gateway no alcanzado.');
  }
};

return (
  <View style={estilos.pantalla}>
    <View style={estilos.filaTitulo}>
      <Text style={estilos.titulo}>Notificaciones {sinLeer > 0 ? '(' + sinLeer + ')' : ''}</Text>
      <Pressable style={estilos.boton} onPress={marcarTodas}>
        <Text style={estilos.botonTexto}>Marcar todas</Text>
      </Pressable>
    </View>
    {cargando && <ActivityIndicator size="large" color="#FFB800" />}
    {!!error && <Text style={estilos.error}>{error}</Text>}
    <FlatList
      data={avisos}
      keyExtractor={(n) => String(n.idnotificacion)}
      refreshControl={
        <RefreshControl refreshing={cargando} onRefresh={cargar} colors={['#FFB800']} />
      }
      renderItem={({ item }) => (
        <View style={[estilos.tarjeta, estilos['nivel_' + item.nivel]]}>
          <Text style={estilos.tarjetaNombre}>{item.titulo}</Text>
          <Text style={estilos.tarjetaDetalles}>{item.mensaje}</Text>
          <Text style={estilos.cuando}>{item.rama} · {String(item.fecha).slice(0, 16)}</Text>
        </View>
      )}
      ListEmptyComponent={
        !cargando && <Text style={estilos.vacio}>Sin notificaciones todavía.</Text>
      }
```

```
      />
    </View>
  );
}


// ---------- App con dos pestañas sencillas ----------
export default function App() {
  const [pantalla, setPantalla] = useState('catalogo');
  return (
    <SafeAreaProvider>
      <SafeAreaView style={estilos.raiz}>
        <View style={estilos.barra}>
          <Pressable
            style={[estilos.pestana, pantalla === 'catalogo' && estilos.pestanaActiva]}
            onPress={() => setPantalla('catalogo')}>
            <Text style={estilos.pestanaTexto}>Catálogo</Text>
          </Pressable>
          <Pressable
            style={[estilos.pestana, pantalla === 'notificaciones' && estilos.pestanaActiva]}
            onPress={() => setPantalla('notificaciones')}>
            <Text style={estilos.pestanaTexto}>Notificaciones</Text>
          </Pressable>
        </View>
        {pantalla === 'catalogo' ? <PantallaCatalogo /> : <PantallaNotificaciones />}
      </SafeAreaView>
    </SafeAreaProvider>
  );
}


const estilos = StyleSheet.create({
  raiz: { flex: 1, backgroundColor: '#F2F3F5' },
  barra: { flexDirection: 'row', gap: 8, padding: 10, backgroundColor: '#14161A' },
  pestana: { flex: 1, padding: 10, borderRadius: 6, alignItems: 'center', backgroundColor: '#2B3038' },
  pestanaActiva: { backgroundColor: '#FFB800' },
  pestanaTexto: { color: '#FFFFFF', fontWeight: '700' },
  pantalla: { flex: 1, padding: 14, gap: 10 },
  titulo: { fontSize: 20, fontWeight: '800' },
  entrada: { borderWidth: 1, borderColor: '#C9CDD3', borderRadius: 6, padding: 10, backgroundColor: '#FFFFFF' },
  boton: { paddingVertical: 8, paddingHorizontal: 14, borderRadius: 6, backgroundColor: '#214F9E' },
  botonTexto: { color: '#FFFFFF', fontWeight: '700' },
  tarjeta: { borderWidth: 1, borderColor: '#D9DCE1', borderRadius: 6, padding: 12, backgroundColor: '#FFFFFF',
       borderLeftWidth: 5, borderLeftColor: '#FFB800' },
```

```
  nivel_exito: { borderLeftColor: '#1E7B3A' },
  nivel_aviso: { borderLeftColor: '#B45309' },
  nivel_error: { borderLeftColor: '#B42318' },
  nivel_info: { borderLeftColor: '#1D5FA8' },
  tarjetaNombre: { fontWeight: '800' },
  tarjetaDetalles: { color: '#4B5563', marginTop: 2 },
  tarjetaEstado: { color: '#1E7B3A', fontWeight: '700', marginTop: 6 },
  cuando: { color: '#6B7280', fontSize: 12, marginTop: 6 },
  vacio: { textAlign: 'center', color: '#6B7280', marginTop: 20 },
  error: { color: '#B42318', fontWeight: '700' },
  filaTitulo: { flexDirection: 'row', alignItems: 'center', gap: 10 },
});
```